LUT University
School of Engineering Science
Software Engineering
Master's Programme in Software Engineering and Digital Transformation

Joonas Heinonen

DESIGN AND IMPLEMENTATION OF AUTOMATED VISUAL REGRESSION TESTING IN A LARGE SOFTWARE PRODUCT

Examiners: Professor Jari Porras
Professor Ari Happonen

Supervisors: Professor Ari Happonen

ABSTRACT

LUT University
School of Engineering Science
Software Engineering
Master's Programme in Software Engineering and Digital Transformation

Joonas Heinonen

Design and implementation of automated visual regression testing in a large software product

Master’s Thesis

2020

70 pages, 17 figures, 3 tables

Examiners: Professor Jari Porras
Professor Ari Happonen

Keywords: CSS, puppeteer, jest, nodejs, chromium, docker, testing, test automation, visual testing, regression testing, visual regression testing, quality assurance, open source

The object of this master thesis is to address the practical implementation of visual quality assurance framework for the layout of the software being tested and to improve the finding of style bugs. In this thesis, a custom visual GUI testing (VGT) framework and a test suite was developed for the target company. The general idea of the framework is to track visual changes and assure that unwanted visual differences are covered before changes end up in production. The main goal of the thesis work was to enable rewriting of the visual layout and decrease the technical debt. The pros and cons of visual regression testing were evaluated with open source tools and frameworks. The evaluation was done with practical implementation and piloting. The outcome of this thesis is a visual testing system and a framework.

TIIVISTELMÄ

LUT University
School of Engineering Science
Ohjelmistotuotannon laitos
Master's Programme in Software Engineering and Digital Transformation

Joonas Heinonen

Automatisoidun visuaalisen regressiotestauksen suunnittelu ja implementointi suureen ohjelmistotuotannon tuotteeseen

Diplomityö

2020

70 sivua, 17 kuvaa, 3 taulukkoa

Työn tarkastajat: Professori Jari Porras
Professori Ari Happonen

Hakusanat: CSS, puppeteer, jest, nodejs, chromium, docker, testaus, testiautomaatio, visuaalinen testaus, regressiotestaus, visuaalinen regressiotestaus, laadunvarmistus, avoin lähdekoodi

Tässä diplomityössä on tavoitteena avata visuaalisen laadunvarmistuksen käytännön toteutusta ja sen merkitystä. Tässä työssä kehitettiin räätälöity visuaaliseen testaamiseen tarkoitettu ohjelmistokehys yritykselle. Kehyksen avulla ohjelmiston visuaalista laatua ja tyylibugien löytämistä voidaan parantaa. Työn tarkoituksena oli mahdollistaa visuaalisen ulkoasun uudelleen kirjoittaminen ja teknisen velan vähentäminen koodikannasta. Visuaalisen regressiotestauksen hyötyjä ja haittoja tutkittiin vapaan lähdekoodin työkaluilla käytännönläheisellä toteutuksella. Työn lopputuloksena oli päivittäiseen kehitystyöhön integroitu visuaalisten aspektien testiautomaatioratkaisu ja testiautomaatiokehys.

ACKNOWLEDGEMENTS

I am eternally grateful for having such a supporting wife and a life partner to share my achievements and the hardest moments with.

I also take this opportunity to express my gratitude to my thesis work supervisors Prof. Jari Porras and Ari Happonen, who showed great guidance and support during the process of creating this thesis work.

I would like to express my gratitude towards the LUT University engineering science department for supporting my thesis work from the academic viewpoint, especially Professor Ari Happonen, who has been a tremendous resource to all software engineering students as continuously driving the level of education upwards.

I'm very humbled to have met such amazing and inspiriting people from the tech industry and academia during my studies and this thesis. Finally, I would like to give credit to all my work colleagues for their patience, wisdom and help.

TABLE OF CONTENTS

LIST OF SYMBOLS AND ABBREVIATIONS

API	Application interface
BEM	Block Element Modifier
CDP	Chrome DevTools Protocol
CLI	Command Line Interface
CPL	Challenges, Problems & Limitations
CSS	Cascading Stylesheets
DOM	Document Object Model
GUI	Graphical User Interface
HTML	Hypertext Markup Language
JavaScript	Most common programming language in modern web development.
Jest	JavaScript testing Framework.
Jest-Puppeteer	JavaScript framework for Jest and Puppeteer.
Jest-screenshot	Visual image differing tool for Jest.
NPM	Node package manager
NodeJS	JavaScript runtime built on Chrome's V8 JavaScript engine.
Puppeteer	JavaScript browser automation framework.
REST	RESTful architecture
RWD	Responsive Web Design
SUT	System Under Test
VGT	Visual GUI Testing

1 INTRODUCTION

This thesis had its initial spark when summer traineeships started in the target company at the beginning of May 2018. Traineeships usually start from bug fixing and getting familiar with the workflow of a software company. The target company was no different in this sense. In the company, it was developer's responsibility to write unit tests and check that your code did not break anything. Breaking means unintentional layout changes, impaired functionality or generally crashing of the whole system. The traineeships in the company included extensive information about the reasons for why one should even begin to test their software. A set of similar values found in the research of web service testing by Sahoo & Ray, 2017. The value of testing was highly appreciated in the company, which was a heavy influential factor for this Master Thesis being about testing and quality assurance.

The company has multiple software components, which are similar in color and style. This results the software under test (SUT) being viewed as a fleet. A fleet can be defined as "a set of systems, sub-systems or components". (Kortelainen et al. 2016) In this case, the SUT has multiple repetitive components, which all require testing and fleet management. As the validation needs and systematic verification grows in the fleet, the system testing becomes more essential, also in the field of visual quality assurance. (Kinnunen et al. 2019)

The general idea of the problem the company had with their enterprise software was that CSS, the styles of the web-application were written without a specific framework, such as Block Element Modifier, BEM (BEM, 2020). When writing styles with CSS the idea is to tell HTML how the web-elements should look and what order they should be in (Takei et al. 2015). In large enterprise software, where the CSS exceeds over 20000 lines of codes, understanding which styles are affecting the elements of the application can be impossible to decipher. Improving the ways of finding the root causes of "style-bugs" and making each developer more confident when writing CSS was a key factor driving the reasons to create visual testing framework for the developers. Porting CSS from no architectural framework usage to a CSS framework will cause serious problems in the layout without proper visual assurance. The web application is stateful, which makes the possible problems grow considerably (Beroual et al. 2017). The problems are seen in broken layouts where UI-components are misaligned, misplaced, hidden or behaving in other unintentional ways. The

visual regression framework built in this thesis will catch these issues and show what exactly broke in the layout. (Mazinanian & Tsantalis, 2017) The UI is built to be responsive, which makes the UI design follow Responsive Web Design (RWD). Responsiveness can also be positive reactiveness to user's requests, meaning also responding to device sizes (Quental et al. 2019). When a site is responsive, it adapts to different sizes according to the device viewport to provide the easiest usage for each device. However, responsiveness also comes with a cost, it makes the UI a lot more complex and may result in responsive layout failures. (Althomali et al. 2019) Visual faults in websites can decrease the trustworthiness of the site, ultimately affecting the overall success of the website business (Mahajan et al. 2016).

Generally, software has not been built from scratch in many years. The same ideology is also true for testing automation. In the modern world, testing automation follows a global trend to digitalize and automate everything that may be possible subject for these actions. The idea is to produce faster ways to bring information to decision making processes, which results to fixing things and developing the solutions further. When testing changes from manual tasks to maintaining automated test scripts, it changes dramatically the roles and tasks the personnel have. Automation changes the whole testing process from the highest level in a company, to affect even on a single employee's own decision making. (Kortelainen & Happonen, 2017) The modern age tools, languages, libraries and frameworks open unlimited possibilities to use boilerplate code and complex functionalities at growing rate (Myrcha & Rokita 2014). Software testing automation has been the industry choice over manual testing due to increased popularity in readymade testing frameworks (Yao & Wang 2012). This was also the founding idea when creating a new custom solution for the company as this thesis work. Web-applications were built typically with HTML, JavaScript and CSS (Choudhary et al. 2010). Currently these technologies are used with packages. These packages are frameworks, tools, snippets or different kind of helpers to give ease of access to different functionalities what the developer may seek in development and web application development. However, using open source packages may result in security vulnerabilities if a developer is not careful (Srinivasan et al. 2017). Depending on the need, sometimes developers must create their own frameworks to achieve something that's not yet available. In this thesis work the built VGT framework is built depending on different secured libraries tools and other existing frameworks to achieve the wanted outcome without a security compromise.

Assuring software quality is an important factor in successful software companies. Software testing can be done manually or through automation (Dobles & Quesada-Lopez 2019). Manual testing can be very time consuming when the system grows in features or lines of code. In this case when the software is a large enterprise software, the views, functionality and layout changes can cascade through multiple layers of the system. Trying to catch these cascading bugs due to the complex nature of the software, manual testing is out of the question, since it's inefficient and ineffective in this case (Debroy et al. 2018). With automatic testing, the complexity of the software can be managed with combination of testing techniques. In the system under test (SUT), the assurance is done with API tests, unit tests on different layers of the system and system testing. Different combination of testing techniques can used in manual testing also. Manual testing is great for small pieces of software. However, manual testing does not catch regression bugs unless the tester tests already tested parts of the software with each new change (Dobles & Quesada-Lopez 2019). For enterprise software, which has a lot of changing different business requirements and functionalities, manual testing is too time consuming and hard. This resulted test automation being the only feasible solution. (Singi et al. 2015) In a set time frame, test automation can run significantly higher amount of different test cases when compared to manual testing.

Faults found with GUI testing can be GUI and non-GUI faults. GUI testing is the end point for the user interaction, "GUI testing represents a form of system-level testing". (Nguyen et al. 2014) To get the GUI in a wanted state the software logic must perform its operations successfully and then the GUI component should display the logic's results for the user to see. Localizations may result in web applications text overflow or break GUI. Test automation is proven to be a solid way of addressing GUI issues. (Alameer et al. 2016) If a GUI test is written to test a case, it tests the views that are shown to end user correctly (Issa et al. 2012). However, due to the functional complexity of the software, using GUI testing as a "goto"-solution to test the logic of a software isn't wise. Creating complex scenarios using GUI components and the underlying logic is much harder and time consuming than writing unit tests for the logic and lighter GUI tests for the UI to look like how it is expected to look like.

1.1 Objectives

In this thesis the high-level objective was to create a framework, which could assure the visual aspects of the software with automation when operated by a professional software engineer. “Some testing tools are designed to be used by professional programmers and test engineers.”, this thesis work aims exactly at that (Sun et al. 2016). The reason behind for test automation was that running tests and trying to catch regressions manually after each code change is not the work a developer seeks or wants to do (Sutapa, et al. 2019). GUI test automation also makes GUI validation more cost effective and lesser needy for human interventions. (Kortelainen & Happonen 2017) Modern web application testing can be troublesome when the application changes and the tests stop working (Hammoudi et al. 2016). This thesis aims to create tests in a way that the software GUI changes do not affect the test scripts at all, making the scripts more robust and sustainable to changes. The objectives of this thesis are creation of a visual testing framework including designing and implementing the testing software and integrating it to the product line and continuous integration (CI) environment. The framework must be able to dynamically generate static views and the data should only possibly change when new code changes are being tested. Two sequential test runs should never end up in different results meaning that the test framework must not have false-positive test cases. In other words, the tests must be effective and valid (Nagowah & Kora-Ramiah 2017).

To create an automated VGT framework, one must first research the available tools and then try the options to find the best ways of doing the visual regression testing. The available tools can be searched from earlier research in the visual quality assurance field. There are also commercial solutions available for visual testing needs, which can be derived to result from developing a custom visual assurance framework being financially risky investment (Percy.io, 2020). However, on the long-term usage of a paid visual testing service like percy.io, a custom framework can be a better choice by offering more flexibility and customizations. Research by Alégroth and Feldt 2014 was based on JMeter and Sikuli, tools to automate GUI testing. Due to the fast evolving of the industry, older testing frameworks are discarded as legacy frameworks in the scope of this thesis for modern Web application testing. The most popular modern web application testing frameworks for JavaScript are Jest, Mocha and Jasmine (Chowdhury 2019). It is also important to understand what kind of

software the framework is testing and how the software works in order to create a usable tool. From the beginning of this thesis the idea was to combine open source frameworks to create an internal custom-framework to handle the specific problems of the development team in the target company. The starting point for this thesis was to pilot existing tools with a smaller software system to find out which frameworks are most suitable for visual GUI testing project, a similar technique used by Börjesson 2012 in collaboration with Saab AB to gain more information about the industrial applicability. Running a pilot project to find more information about the tools is a known technique for software research (Al-Tarawneh & Althunibat 2019). The pilot was completed successful with Jest and Puppeteer - frameworks and the results were promising. A working MVP, which produced measurable visual artifacts from a real software product. The general idea was to see in practice what kind of problems and key factors should be considered in the design choices of larger visual quality assurance framework.

When doing testing, the test tools are quite limited to the environments one can test in, especially in web applications (Negara & Stroulia 2012). To get a system in a specific state, the tools to achieve that usually include automation testing framework knowledge in addition of knowing the software which is being tested (Sharma et al. 2018). At the start of this thesis work, the architectural knowledge of the SUT was quite poor as expected in any software that is developed by others, which resulted objectives being harder to specify and manage.

In the current state of software development, new frameworks, tools and usable artifacts are published or updated daily and are used to create and maintain software used by millions of people (Hanna & Jaber 2019). This results the research data of frameworks being valid for only a limited time. One of the most important factors of selecting open source software to use, is the popularity of the software. When a software is widely used on private and public companies, it means that the software has longer expected life-spawn (Chowdhury 2019). When a software is expected to be supported and developed in the future it becomes a less of a risk and a better choice for the long term. The chosen frameworks for this thesis are initialized by big companies like Facebook or Google, which make them very popular among developers in open source communities such as GitHub. (GitHub 2020)

In this thesis the tool selection objectives were the following, long-term maintainability, up to date documentation, open source and ease of use in development. With these objectives the selection of testing frameworks included Jest and Puppeteer. Both repositories are popular and used widely (GitHub 2020), which was also approved by the development team as end customer of the internal framework.

1.2 Limitations

Visual regression testing is about having an artifact to extract information about the visual state of system (Weltzmaier et al. 2016). In visual testing there is known limitations due to the nature of image processing and complexity of GUI. Visual GUI Testing (VGT) can be configured to catch all differences, which are not exactly as wanted from colors and pixel differences. (GitHub, 2020) However, other aspects such as how usable the view is technically, is not assured by VGT. To assure how usable the view is technically, an image stays immutable when new code changes are merged and affecting that image, a comparison method must be defined. When assuring that layout, buttons, views, tabs and all visual elements are on their correct places and work as intended. The most common options for doing the comparison is: 1. Going through every view by hand manually and assure that nothing has changed compared to the previous version. 2. Doing the comparison through software automation. The automation can be done by Creating a baseline image for each view and comparing baseline with a new image which is affected by code changes. This thesis is based on automating the second option. In addition of the two proposed choices a lot of other techniques and methods have been proposed to make the quality assurance faster and more effective through automation (Simos et al., 2019). However, in some cases the test automation does not accelerate the assurance work but shifts the manual work to creating test scripts and doing script maintenance work (Weltzmaier et al. 2016). Reaching full automation testing coverage in numerous GUI paths is not targeted. The work in this thesis is limited on core features and views of the tested software, to have the VGT framework on a reasonable level of testing and not greatly exceed the time limits of this thesis work (Adamoli et al. 2011).

When creating a framework for testing, the key factor is to understand why to write tests in general. Assuring the quality of a software has a lot of different aspects to consider. In test automation, the more complex a test case is, the harder it is to create a simple and usable framework to serve all different test cases. (Nishi 2015) The complexity of test cases may come from many places, but in practice the complexity may depend on everchanging business requirements which are not usually known at the start of the project. When a project has a lot of different requirements, automated test generation can't be used. In this case the test case generation requires intensive knowledge of the SUT, resulting in a need to write the test scripts and test cases manually (Dadkhah 2016). In many tools used in the testing automation industry, manual scripting is the most common way to enumerate test cases and define them (Mahajan et al. 2016). In the context of this thesis the scope and requirements of the custom framework were known and detailed by developers. The limitations of Jest, Puppeteer or any other chosen framework can be found from the GitHub issue boards (GitHub 2020). There's a lot of different feature requests, open issues and pull requests. This means that the dependent frameworks are constantly evolving and changing. However, there's always a stable release available, which offers a static dependency tree. The most recent stable release was used from each external framework, which made the limitations management easier.

It is necessary to understand a concept by explaining it through different levels of abstraction, like many researchers have done (Nguyen et al. 2012). The framework uses a headless browser, which can be triggered from a Command Line Interface (CLI). The tests are running in NodeJS and are written with a testing framework called Jest. From Jest the test cases are joined to Puppeteer, which controls a headless browser. A single test sequence can be shown as follows: NodeJS starts, Jest gives instructions to Puppeteer, headless browser follows the instructions and returns a result to Jest, which interprets the result and starts the next test case sequence or sends a signal to exit NodeJS. Testing frameworks usually include a kind of guiding architectural idea, which divides the testable functionalities in a few different definitions. To understand a general idea of testing frameworks one must understand how these frameworks are used (Nishi 2015). Roughly these frameworks have test suites, which run test cases, which expect a result to be the expected result. If the result is not expected result the test fails. (Jest, 2020) Jest is a testing framework, which is used in

this GUI tester project to wrap the test-flow in common understandable and usable way for the end users of this thesis, the target company's UI-team developers.

The general usage of the framework needs always an URL to navigate and test. If a view does not have an URL to navigate to, then the view can't be tested without additional configuration. However, some views might have tabs, buttons or other controls, which affect the outcome of the test. To include these kind of test cases, a need for element specific selector was created. The limitation in tests, which emulate clicking a button or a tab, might result in failure if the element selector or identifier changes, also resulting in growing script maintenance work. Overall the test cases are limited by the SUT's views that should be tested. The SUT uses local storage to handle UI's stateful components among the user's information, such as access rights. Local storage is common way to store data about the current session or the user to the web browsers memory (Anand et al. 2015). In this case the framework needs a way to change the local storage data between tests. Local storage is shared within a browser, which makes eventually parallel test runs impossible due to the lack of virtualization resources, computing resources and the nature of the tested software, ultimately compromising the image results of the VGT framework. Image captures are affected by the rendering speed of the CPU or browser, animations and most importantly: environments. For example, the fonts in a browser differ when the operating system changes between Linux, Mac and Windows. Other differences in the base images than the actual changes affected by code changes are not wanted. In addition, handling animations and consistency between the image comparisons are known limitations (Alégroth & Feldt 2014). The tests performance is limited by CPU and RAM. How fast a computer is, affects the speed of rendering and running JavaScript in a web browser. If the computer is slow, the rendering might be incomplete, making the test cases fail randomly resulting in false-positive cases for image comparison.

Continuous integration (CI) tools help developers to test their code as early as possible (Ebert et al. 2016). Integrating the test framework to a CI environment is limited by the CI-environment configurations. For example, running tasks on a specific sequence may end up with different result in CI environment than on local machine run. The differences are caused by multiple layers of configuration including network, performance and task sharing. Running tests on a local environment can also cause heavy load to the desktop or laptop

resulting other development practices suffering. (Rahman et al. 2015) The CI Tools have increased in industrial popularity, since the need for faster delivery and high software quality are demanded by customers. CI tools are more commonly used to run automated unit tests to gather and deliver rapid feedback, since the academic literature has “a gap regarding empirical, industrial, research on the use of higher-level, and in particular, GUI-based testing in a CI environment”. (Alégroth et al. 2018)

1.3 Structure of the thesis

This thesis consists of four key areas. First this thesis covers the current state of visual testing and automation testing. When considering different ways of doing visual testing or test automation, a precise look of available tools and research based on the usage of these tools, is concluded. Multiple different visual testing solutions are created and iterated, which indicates that a need for constant improvement is ongoing subject in the industry (Wetzlmaier et al. 2016). The obvious reason for constant improvement, the emergence of new frameworks and ways of doing visual testing originate from not having a solution which is superior to others and due to increasing complexity of web applications (Yousaf et al. 2019). The complexity of GUI testing varies by systems size and assurance requirements resulting GUI testing in a constant challenge (Wetzlmaier et al. 2016). Also, when testing a component-based system like SUT, the assurance work poses additional challenges, which are elaborated in the later parts of this thesis. (Khan et al. 2012) In the second part of this thesis, a dive to the history of visual testing frameworks is concluded. The history section includes analysis of problem cases during the need to assure visual quality of software. In the third part of this thesis an in-depth analysis of the selected testing libraries and frameworks is exhibited. In-depth analysis goes into more details of the technical implementation and experiences during the development of the custom VGT framework. In the last part of this thesis, a look to the differences in large scale software projects may include and how that affected this thesis work.

2 THEORETICAL BACKGROUND

In this chapter, a closer theoretical look to tests automation and visual regression testing research is carried. Additionally, the benefits and defects of visual testing, maintenance and quality assurance are being discussed. In the following sections test automation and manual testing are compared and the current state of visual testing research is reviewed. The methods and learnings of previous test automation and visual test automation studies are used to showcase the state of the research, also from the quality assurance and maintenance point of view.

2.1 Test automation

The fundamental nature of testing has two options, manual or automated. When going deeper to the field of testing, the software under testing is usually the defining factor how the testing should be done. In the field of large software projects where the budget is in millions and the userbase is vast, usually the software is quite complex. In these cases, manual testing is out of the selection scope since it becomes financially a too heavy burden to carry out for the company (Lenka, et al. 2018). Automation testing makes tests run faster and decreases the manual effort in testing making automation testing more profitable in longer scope. (Börjesson & Feldt 2012) In large software projects the quality assurance is usually divided to groups depending on the nature of the software. Usually the quality assurance teams have specific objectives based on the part that they are responsible for assuring (Alíc et al. 2017). Generally, a software has some logic, which is the essential part of the software defining itself as working. In addition to logic the software usually has a kind of interface to interact with. These interfaces can be of multiple types, for example application programming interface (API), graphical user interface (GUI) or command line interface (CLI). Every part, aspect and permutation of different visual software configurations can be tested (Selay et al. 2014). The more types of tests a software system has, the more reliable, stable and maintainable the software is (Coppola et al. 2019). With test automation, different types of tests can be written to create more stable software product.

A single test case is usually quite effortless to create and execute manually. To concretize, let’s imagine test cases A and B. The test case A could be for example a case, where clicking

a button should open a modal for the user. If the modal is opened successfully the test operator marks the test completed successfully and continues to the next case. Now if the next case B causes the button, which opens the modal to fail, a manual test operator has no way of catching that regression but to test all of the test cases in different orders to make sure that there's no regressions. In this example, catching regressions requires first making sure each test is passing individually as A or B. After both cases are successfully completed the next step is to test both cases sequential in the following order A + B and in reverse order B + A. When the amount test cases grow, the combinations and possible regressions grows also, making manual testing a risky choice financially and workwise. A study by Alégroth et al. 2018 shows that manual testing is cheaper for the first 44 weeks in their study at hand but after that automation testing is a better choice financially speaking. From the study somewhere between 40-50 weeks can be as a point to break even when comparing manual and automation testing financial aspects. Also, when considering the long-term automation testing, the financial breakeven point should come in a shorter period than 40 weeks by utilizing earlier work and knowledge. The amount of work to create a runnable automated test is dependent on the complexity of the test cases that need validation, in this case creating automated test framework to support just few tests is a choice of no-one. However, in larger amounts of test cases the automation framework and test runs start to pay off. Therefore, the known limitations of automated tests usually include a high initial investment and the hardship of generating the test script with a selected tool. (Moura et al. 2017)

2.2 Visual regression testing research

The fundamental nature of visual quality assurance is about understanding the reasons why and how visual defects end up in production software, which requires a competence in testing automation (Savchenko et al. 2016). Visual defect can be defined as an unwanted and unintentional graphical change of a GUI component or view. Through automated testing processes regressions can be mitigated. (Ozcan 2019) Bugs, faults and software hinderances are caused by time constraints, faster time-to-market or other requirements involving the development team to take shortcuts for the cost of quality (Börjesson & Feldt 2012). Software has been created by humans, even software which creates more software, is still strongly affected by human touch resulting in unseen behaviors. If one can understand how visual defects are formed, one can try to find a feasible solution to the root cause. Since the

human-factor cannot be taken out of the equation, the only way is to mitigate the risk to make errors.

The challenge of visual defects is in the variance and complexity of the software system. GUI is the endpoint for user interaction making it a real goal for GUI and system testing. (Alégroth et al. 2015) The amount of different possibilities in a software system can be seen by the definition of the “soft”-ware word. Soft, tangible, modifiable, the essential substance of software, which makes software, software. This is one of the key reasons why visual quality assurance is so difficult, due to the software changing. (Panda & Mohapatra 2014) The tests and precautions must try to take as much as possible software changes into account to decrease the workload in a long run (Mahajan et al. 2016). In software development, a lot of tools are available, which help developers to make less mistakes when programming. These tools are preventative methods for quality assurance, they guide the developer to code more consistent and errorless code. However, the tools are limited by the scope of their knowledge about what problems can be caused by what change. The tools might help by fixing syntax or displaying error on bad coding practices. Some of these tools are called Linters. ESLint is a JavaScript Linter, which is used also in this project to write more consistent code. Linters are style guides, which formats the written code on runtime and may suggest how exactly the code should be written correctly. (ESLint 2020) However, linters have little to no-clue about how correct the code is in the scope of the software. Since there’s no easy way of configuring an effortless and performant runtime check if the software is producing the results it’s expected, development and visual quality assurance needs to be done separately.

The field of software testing research is vast. A lot of different research in the field of visual software testing has been conducted (Stocco et al. 2014). Usually research topics vary from examining the behaviors of the software developers or teams to in-depth technical analytics and mathematics. From the vastness of software testing research, one can argument that there’s an endless area of problems to research. One of the key factors affecting software testing is the speed of technological advancements (Yao & Wang 2012). The speed of new emerging technologies and tools is accelerating constantly, which results also the acceleration of testing software development and advances (Ramler et al. 2018).

In software development the nature of continuity results in the need of tracking the technical aspects of software development. A tracked change can be anything, that is related to the definition of the software being exactly the software that it is known for. A view, button, core functionality, substance of a software, anything that can define the software as it is. Defining the validation of each GUI component or object can be troublesome. (Navarro et al. 2009) The tracking of changes results the changes being either wanted or not. In the case that the tracked change is not desirable or a new feature, it can be labeled as a regression. Regressions are unwanted changes, a side effect of a new feature affecting aspects of the software system accidentally. (Panda & Mohapatra 2014) Since the types of regressions is vast, this thesis focuses mainly on visual regressions.

Image differing is a computer vision-based technique to determine visual changes in image comparison (Adachi et al. 2018). This thesis does not take part in the creation of the image differing framework, since there's plenty of plug-and-play options available as stated in a study by Tanaka 2019. The available tools can be compared for their usability, performance, stability and configurability. In this thesis work, two available open source tools were compared. The discussion of the tools comparison results is in the later parts of this thesis.

In a software system that has multiple layers and views, a strategy to create useful and insightful test cases is essential. Test cases and testing strategies vary from the objectives one aims for. (Yildiz et al. 2012) In the case of this thesis, the objectives are related to creating a wholesome solution to serve the needs of information about the visual state of the software.

2.3 Benefits and defects of visual testing

Visual testing has been part of quality assurance for as long as there has been software with GUI. In visual testing the need to find the most performant methods to find software faults has resulted in automation (Shin & Bae 2016). The reason for test automation being the driving factor for more quality assurance teams doing visual testing, is that manual visual testing is very laborious in the long run (Börjesson & Feldt 2012). Using automation tools on visual testing needs saves effort and time if the duration of the project is projected to be

long. However, the initialization requires more time to build the infrastructure to run and create automated visual test cases (Alégroth et al. 2015).

In a study conducted by Alégroth et al. 2015 a vast selection of challenges, problems and limitations were found on two industrial cases in Sweden. The themes of challenges problems and limitations (CPL)s were test tool deficiencies, image recognition, test specification, SUT testability deficiencies and Script tuning. In the test deficiencies the API documentation had flaws, which resulted in confusion and frustration among developers, ultimately hindering the feeling of superior quality assurance. In Image recognition, a need for scripts to handle tricky test cases and the exceptions indicate that more research is needed for image recognition algorithms to be more lightweight on the performance. Script tuning was found to be the most time consuming and frustrating source of CPLs. The CPLs caused from script tuning can be mitigated with more experience using the automation testing languages and techniques. The other factors mentioned are not seen as applicable for this thesis work. (Alégroth et al. 2015)

Visual testing can have different types depending on how the testing is defined. The optimal case would be that each test run is done in development environment, which assures that the visual defects won't end up in the production software. This kind of testing can be preventive. Preventive testing is usually the choice of testing, since the software needs to be assured and tested before it is published to production environments. Testing can be also defined and specified as set of use cases to define the behavior of GUI to be tested or as a set of annotated elements of GUI (Navarro et al. 2010).

2.4 Software maintenance & Quality assurance

Software maintenance and quality assurance are close to each other. Maintenance work is essential to do for quality assurance. And without quality assurance the maintenance tasks are limited to the knowledge of the maintenance engineer. Routine maintenance work includes assuring that the software works in general, measuring different aspects of the software system and taking actions if any faults are found in the system. Open source systems have made it possible to have researchers analyze better how does the maintenance work and quality assurance intervene with each other. (Banerjee et al. 2015)

One of the maintenance work tasks is to prevent faults from happening by upgrading software systems to newer versions, which are more secure and robust. If the software system source codes are written in a hurry or with inexperience, the software may have faults. When a fault or a bug is found in the system, the software maintainer must have a plan how to deal with the situation. In open source repositories, a bug tracking system and usage documentation is available for the public. Usually the maintainer validates and tries to reproduce the fault or bug to confirm that the software fault is correctly reported. When a fault has been confirmed, the fault must be fixed. To fix a fault it must be first identified from the source code and possible fixes must be tested. To identify the fault from the source code the maintainer must know the software system inside out. To know a software system inside out, either your one of the developers of the system or you must reverse-engineer and understand how the system is built in order to maintain it with a GUI framework (Amalfitano et al., 2015). The tools to understand the software system or gaining knowledge of it, is done by exploring and tinkering the software in a safe environment. Sometimes the software systems are too complex to understand how they work. By debugging a runtime version of the software, one can understand how the software works. (Banerjee et al. 2015) In GUI testing, debugging a single test case can be very laborious if the used tool doesn't support recording the test execution or being able to show the actual test execution on runtime. (Pham et al. 2013)

One of the major issues in automation testing is test script maintenance. When the application changes and breaks the test cases, a developer needs to manually rewrite or refactor the affected test cases in order to get the automation tests running if the application is working correctly. (Imtiaz et al. 2019) Having broken test cases in test automation suite may affect other test cases, making the whole test suite unreliable. Yet, depending on the architectural choices and test cases, there is a possibility to write tests that are isolated from having any effect on other test cases in the test suite.

3 DESIGN PHASE

Before the actual design and plans how to create this thesis work technically, a lot of research content were consumed in the areas of software testing, quality assurance, visual testing and visual regression testing, which is common to any research paper (Sahoo & Ray 2017). The whole picture of software visual testing opened in a different way one could expect. The initial thought of visual quality assurance was that, it's quite straight forward. After one's being affected by academic literature about testing and quality assurance the picture of testing could be defined as endless multidimensional challenges in software systems. From this thought, the idea of creating architectural plan was to make it simple and understandable, since simplicity is valuable for practical usage in many cases (Happonen 2011).

A brief mapping study was conducted from academic and non-academic sources to find if current available tools were able to deliver the requirements of this thesis. At the time the technical implementation work of this thesis was initialized, the current tools weren't enough for the needs of the company. Therefore, designing a framework and a test system is the goal of this thesis. The designing work started from the requirements of the goal. From questions why and how. What is needed to have a working and usable testing system. The first designs of the system were created with a member of the company's UI development team. In the first designs the guiding values were tested by writing non-working code to display how the end results would work. From the placeholder pseudo codes, the general requirements of the system were quite easy to understand. The outcome was that the system should give a highly maintainable and effortless way of writing and running tests that assure the visual quality of the software being tested.

Extensive amount of literature has been produced concerning the popularity of digital solutions (Viswanadha et al 2019). The challenges of gaining and overall understanding of visual quality assurance in the context of web-applications is troublesome (Hanna & Jaber, 2019). The academic literature had little to no materials on recent practical reports of visual quality assurance tools including Jest or Puppeteer. The most recent and closest work to this thesis was created using Jenkins and JAutomate, an effort to run visual GUI testing (VGT) in CI-environment. The study shows that creating and running GUI automation in CI-environment can be beneficial. (Muhamad et al. 2016) The research of current open source

quality assurance tools had to be very extensive in order to bridge the gap between new open source tools and academic literature. A lot of articles, tutorials and video-content was seen helpful for better oversight of the capabilities of currently available tools and frameworks for modern web application testing.

When working in a software company with an additional project, the time that can be taken from the developers of the money generating software is limited. Constant hurry and strict schedules were a huge challenge during the whole journey of the technical implementation work of this thesis. Most importantly, the design phase was not seen as too valuable for the developer team to put time into pondering about how and what should be done in this thesis work. Since visual quality assurance was fairly unknown domain for all participants of this thesis work, the outcome of design phase was evident as inadequate.

The designs of the framework and test runner started by researching the available content from academic and other sources. The overall view of what the framework and test system should be able to do was distinct, but the lack of practicality in academic reports or other sources concerning the selected tools resulted in pains and headaches in the actual technical implementation. In the design phase many of very important details were unknown. The unknown details were discovered during the implementation phase and the designs and fixes to emerged problems could have been avoided from the beginning with more time from all participants, especially from the experts. Also, a search for a mentor with experience in VGT or the selected technologies could have been extremely helpful for the design phase.

The requirements of this thesis work were approved with the development team members, who also are the end-users of this thesis work. The initial requirements were to mock server values and login authentication for performance and stability reasons. Overall a framework to assure the visual quality of their software with as little maintenance as possible. During the implementation new requirements were added: mocking date, creating communication between SUT and the testing framework, and these changes should be done with minimal changes to production code. With the new requirements and the initial ones, the hidden requirement of this thesis work was to reverse-engineer the whole product to understand how it handles data and communication with various components. However, it is unfortunate, that the requirements and capabilities of usable tools for the thesis work were a bit vague

due to constant hurry in the demanding business world, yet understandable. The blurry understanding of SUT, emerging new and hidden requirements ended up affecting a lot on the timeline, which the thesis project was completed.

3.1 Visual testing architecture

Every visual testing architecture has baseline requirements, which the testing framework should cover. Usually architectures also have very deep underlying problems that they want to solve (Viswanadha et al. 2019). The testing architecture should have a way of testing the visual quality of the software, running tests in local and cloud environments but also being able to create tests. In this thesis work four configurable test frameworks were selected to form an internal visual quality assurance and testing framework, a VGT framework. The key elements of any visual testing framework include understandable test-flow, control of the visual materials creation, usage of the created visual materials in test and the results displaying (Kiranagi & Shyam 2017).

The overall architectural need of this thesis is to either select a ready, but configurable framework or to create one from scratch or by combining libraries or frameworks. The needs of creating a visual testing framework for a company is not unique, there's been successful commercial implementations of some visual testing tools in the industrial setting (Alégroth et al. 2014). However, complete ready commercial visual testing tools for sale were not considered in this case, since they were not fit for the customization needs of the company or this thesis work. The architectural parts of the visual testing framework consists of a way to model and manage the tests, a tool to control browser to generate visual artifacts such as screen captures of the browser, a library to make test-flow and image-capturing communicate with each other and lastly a framework to test the visual comparison in addition of displaying the results. This thesis work is created by configuring readymade open source frameworks and combining them to a custom framework, which uses readymade and configurable open source frameworks as the main architectural parts.

There are a lot of readymade open source test frameworks (GitHub 2020). The framework options use quite similar test-flows compared to each other. Usually each web application compatible testing framework has methods to create a suite, test block and a case, which can be parametrized as Xie et al. 2016 have pointed out in their research. A suite is a collection

of test blocks, which consist of test cases. A single test case tests a single state of the system. The usual idea is to expose the functionality to the testing framework and to use it to expect wanted results. However, in VGT the testing activities do not usually care about the SUT's source codes, but rather functionality from the end users point of view. In visual testing the expectation of the test results usually includes a comparison of visual artifact such as a screenshot (Muhamad et al. 2016). Screenshots can be taken also without exposing the functionalities from the software under testing, which is the case of this thesis work. The frameworks have methods and configuration options to make something happen before the tests are ran. The test preparations are essential part of the testing framework, since the tests may need a constant state, a resetting or called functionality to run in.

3.2 Tools and frameworks

In the context of this thesis, where the aim is to assure the visual quality of the tested software, a way to capture the state of the system is needed and a way to compare the changes to the captured state of the system. In other words, this means that the selected tools and frameworks should be capable of capturing the state and changes of SUT. The visual aspects of a system are tested usually with comparing a baseline screenshot with another screenshot, which is affected by code changes. To gain screenshots of a webpage a need for a framework to control a browser is valid (Ramya et al. 2017).

Screenshots comparisons are one of the most common modern visual testing techniques (Alégroth & Feldt 2017). The idea of the screenshot is to capture each element of SUT from SUT's views that tests the visual aspects of the elements found in the screenshot. Screenshots can be compared with other screenshots and the results provide reports of the state of the system. The screenshots from the system can be generated manually, but automation saves a huge load of time and effort. A study by Alégroth et al. 2017 shows that using Sikuli, a python based VGT tool, it is possible to use visual resources to find rare bugs. However, in web applications screenshots are usually generated by automation software that controls the browser. The most common open source frameworks for browser automation are Puppeteer and Selenium (GitHub, 2020). Selenium being very popular and used in academic research (Selay et al. 2014).

Selenium is an automation framework for browsers with multi-browser and multi-OS support. Selenium specializes in testing in multiple platform and environments, but this comes with a cost. Selenium is a lot harder to configure compared Puppeteer. Selenium is harder to configure, since the number of platforms and environments Selenium is supporting is high. Selenium also supports a feature called Selenium Grid. Selenium Grid works as a hub for connecting different platform and browsers to run test suites via web-sockets. Web-sockets are a good way when the need is to have a broader view of the system quality. (Ramya et al. 2017)

Puppeteer is also a browser automation framework, but it is way easier to pair with testing frameworks like Jest when comparing to Selenium. Selenium has known issues in performance and usability (Shariff et al. 2019). The ease of use is a key factor for maintainability point of view. A framework which is dependent on multiple browser and platforms has a higher risk of faults. When running image comparisons in different browsers the HTML and CSS may produce entirely different results on different environments (Selay et al. 2014). Puppeteer can be toggled to run as a headless browser or a visible browser. A headless browser acts like a command line program, it doesn't open a visible browser instance making it easier to integrate to pipeline-environments. Being able to toggle to visible browser instance helps debugging and writing the test cases. Due to higher maintainability, ease of use and easier CI-integration Puppeteer was selected. This thesis won't go into deeper detailed analysis about comparison of the two frameworks, since it's not relevant in the scope of this thesis.

In this thesis, the custom visual testing framework needed a solution which could communicate with Puppeteer in a NodeJS environment. The most known options for test frameworks which control the test structure are Jest, Jasmine, Cypress, Mocha and Chai (Chowdhury 2019). During the comparison of these frameworks the aspects of evaluation were performance, syntax, documentation, support, ease of use and most importantly integration with Puppeteer. The most performant among the criteria was Jest. The most common and documented framework for Puppeteer. There was already a library called Jest-Puppeteer, which offered a preset way of configuring Jest with Puppeteer. A bit of research and actual tryouts on the visual testing projects using Jest and Puppeteer and it was evident that Jest and Puppeteer work great together with the preset.

In this thesis work the chosen testing framework was Jest. Jest is developed and maintained by Facebook open source and is widely used and known. Jest has the usual features such as high configurability and easily understandable test-flow. Jest runs in a NodeJS environment, which makes it easily configurable to other libraries and of course Puppeteer which uses NodeJS as their environment of choice. (GitHub 2020).

The only missing piece of the puzzle was a visual screenshot comparison framework. The ready options for image comparison were Jest-image-snapshot and Jest-screenshot. The comparison reasons were very similar to the test frameworks and Jest-screenshot was the selection of choice. The biggest reason for this was an automated HTML-reporting after each test run, which excelled in displaying the changes clear and simple to developers.

The more layers the custom framework has, the harder it is to maintain. Having a complex framework with multiple moving parts, testing is utmost important and necessary (Garg et al. 2015). As with the company's software which, already had a lot of unit tests to test the functionality, the custom framework also needed unit tests to assure that the framework didn't break on its own. Jest was selected as internal unit-testing framework for the custom visual testing framework. Since the complexity of the custom visual testing framework grew by each readymade configured framework, a lot of work had to be done to assure the longevity of the thesis project.

Tool	**Type**	**Selected**
Puppeteer	**Browser automation**	**Yes**
Selenium	Browser automation	No
Cypress	Browser automation	No
Jest	**Test automation**	**Yes**
Mocha	Test automation	No
Chai	Test automation	No
Jasmine	Test automation	No
Jest-snapshot	Image-comparison	No
Jest-screenshot	**Image-comparison**	**Yes**

Sikuli	VGT tool	No

Table 1. Selected tools.

3.3 Browser automation

To generate the tests the VGT framework had to accomplish authentication mocking, server mocking, date mocking and taking the screenshots at first. These tasks were created by using browser automation, a strategy to automate test flows using scripting and automation tools (Joy & Singh 2015). The tool of choice was Puppeteer, a framework developed and maintained by Google (GitHub 2020).

Puppeteer runs on Chromium, an open source project that forms the basis for popular browser Google Chrome. The difference between Chrome and Chromium is that Chrome is updated regularly by Google and Chromium is not. If a layout works on Chromium it will work on Chrome too, which is a key reason why Puppeteer is a great tool for visual testing. (Puppeteer 2020)

Puppeteer allows developers to open browser instances, tabs, mock clicks, typing and almost anything that is possible within a browser. In this thesis work the most important functionality for the design phase, was running external JavaScript code in a Chromium browser with *page.evaluate* -method. Without the ability to run external code in a browser, the implementation of the framework becomes very troublesome for various reasons. Puppeteer has also a huge library of methods to control things like network. With Puppeteer a lot of information about the state of the browser and data are easy to modify. (Puppeteer 2020)

Puppeteer has a low learning curve to understand the basics, but it is hard to master like programming in general. Compared to manual work, Puppeteer allows developers to write automated tests running in a headless browser with relatively small amount of effort. Creating scripts with Puppeteer require adequate knowledge of Puppeteers domain.

However, maintaining puppeteer scripts and understanding how the browser layers work need knowledge of DOM in order to create long lasting solutions.

3.4 Techniques

Testing has a lot of types and forms, that it can be done in. The most known testing strategies include unit testing, integration testing, system testing and acceptance testing. Unit testing is testing the very basic unit of the software application and is categorized as white box testing. Integration testing combines different parts of the software and ensures that different parts of the software works together. System testing tests the whole software as a complete system. Acceptance testing is done by the users or customers of the software as main goal ensuring that the software works as intended. (Sneha & Gowda 2017)

Testing types can be divided into white box, black box or grey box -testing. In white box, also known as clear box testing, the software's source code and code structure are known and tested. White box testing defines test cases based on the functionalities found in the source code. Black box testing being the selected testing type of the thesis, does not care about the internal parts of the software but is more interested about how the system behaves and displays correct output. Grey box testing is between white and black box testing. In grey box testing the tester has some knowledge of the SUT but does the testing as black box testing, focusing testing on the output of the system. (Sneha & Gowda 2017)

GUI automation techniques can be divided into three "chronologically delineated generations". 1st generation uses exact screen coordinates, 2nd has access to SUT's GUI Components and 3rd (VGT) is based on image recognition. (Alégroth & Feldt 2017) This thesis work is based on using the 3rd generation VGT image recognition as the verification method for finding visual faults in the SUT.

This thesis uses black box system testing with VGT image recognition as the chosen techniques. VGT cares about the state of the system's views and their behavior but not about the system's internal mechanics and codes, hence black box testing. The main idea is to ensure that the system works as it should for the end user, customer. This thesis narrows VGT testing to ensure that the main views and core functionalities of SUT provides the

wanted outcome consistently. On failure the system reports the failed tests and visual differences on system testing level in a visual report tool.

Technique	**Type**	**Selected**
White box testing	Source code based	No
Black box testing	**Cares about system output**	**Yes**
Grey box testing	White & Black combined	No
Unit testing	Single code unit based	No
Integration testing	Unit tests combined	No
System testing	**Complete system based**	**Yes**
Acceptance testing	End user based	No
1st Generation GUI testing	Screen coordinates	No
2nd Generation GUI testing	GUI Components	No
3rd Generation GUI testing	**Image recognition**	**Yes**

Table 2. Selected Techiniques

4 IMPLEMENTATION

The implementation of this thesis followed the company's own agile framework, which is a combination of Kanban and Scrum. An extended Kanban board with Backlog, To Do, Doing, Review and Done columns was used during the project, a standard digital Kanban built into Jira (Nakazawa et al. 2017). The project management tool used in this Thesis work was Jira by Atlassian. Jira had integrations to TeamCity and Bitbucket, which made the workflow easy and efficient.

Before the actual work began, an initial meeting was held to discuss the thesis work objectives, documentation and timeline. From the first meeting a plan to track the thesis work was formed. The plan consisted of a cycle, which had the following points: a meeting to discuss the next steps of the thesis work, a creation of a plan to fulfill until the next meeting and implementation of the features described in the plan. The implementation followed the mentioned cycle and repeated it until the work was finished. During the cycle, occasional help, clarification, analysis, comments or other kind of advice were given by the senior members of the development team.

4.1 Environment

In software development environment configurations and configurations in general can have a dramatic impact on work effectiveness and to the delivered results (Winkler et al. 2018). In the company, the usual code-editor for UI-developers is WebStorm. WebStorm is an advanced IDE created by JetBrains, a respected company among developers in general. This thesis work implementation also used WebStorm as the IDE of choice.

From the design, research and requirements, setting up the environment included a lot of software installations. Among the installations were the obvious tools for any developers who care about version control. The installed tools were Git and SourceTree. A default package manager, NPM is bundled with the installation of NodeJS. NPM offers the ease of adding, removing and managing the packages to any NodeJS project (Npm, 2020). The usage of NPM was in a huge role, since it installs the required packages to get the project running.

In software there's usually multiple ways of getting the same outcome. The differences vary from robustness to performance, but the outcome is still the same. A senior developer knows from experience, which way of implementation follows the best practices. A junior developer struggles to get things working and might be happy after a software seems to be in a stable state. The lack of experience ended up wasting time on bad implementation, which were removed or refactored during the later parts of this projects. Luckily the bad implementations were caught on time by the development team members during presentations of the state of the implementation. From these presentations the knowledge capital grew and enabled better solutions to future challenges.

Using NPM to install the GUI-Tester project dependencies was the easy part of environment setup. The harder part was to understand how to configure Jest, Puppeteer, Jest-puppeteer and Jest-screenshot to work together. In addition of getting the four premade frameworks to work together was to understand how to add own files in the use of these frameworks in a NodeJS environment.

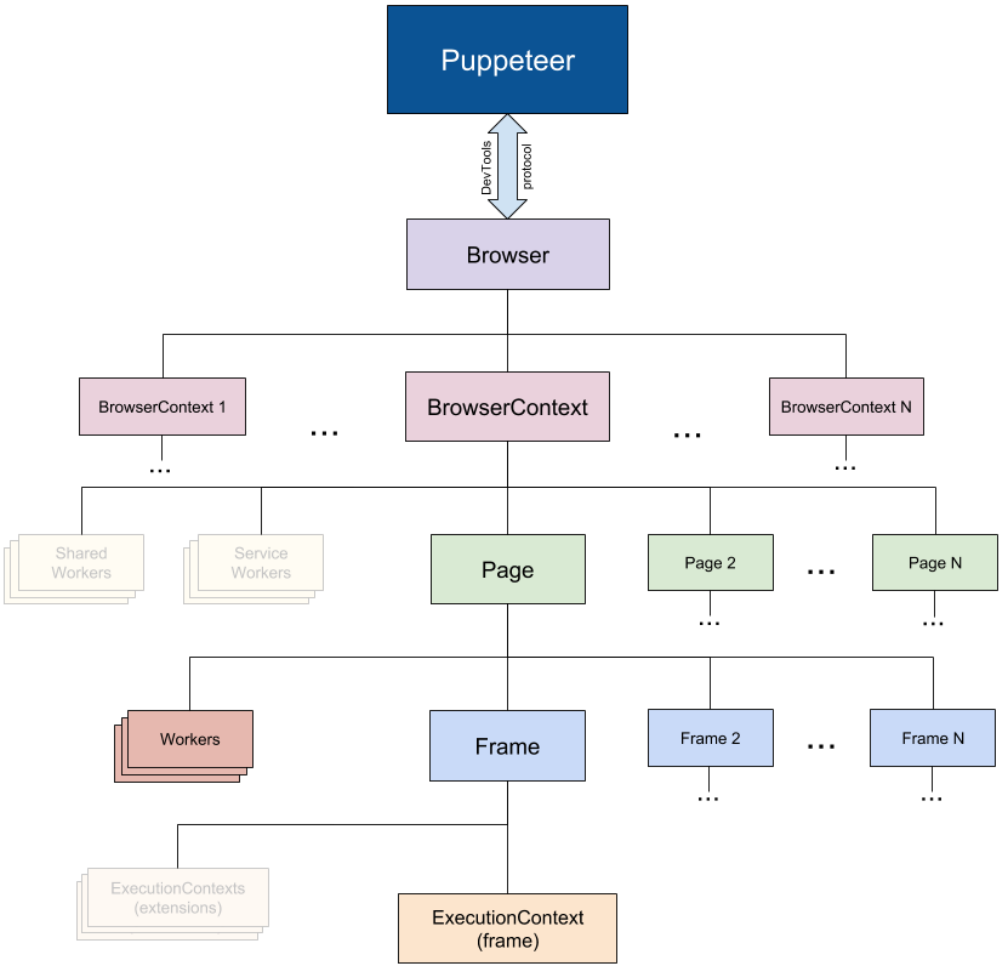
In the later parts of the implementation, the problem of font rendering was found. The problem description is that when developers uses the thesis work in different environments, the screenshots differ. This means that if a Windows, Linux and a Mac user uses the VGT framework together the pictures are constantly changing and unreliable source of assuring the visual quality of the software. To fix the environment dependent usage, a discussion led to "dockerize" everything with Docker. The differences in screenshots between environments were tested in five different Windows 10 environments and in one MacOSX both running the tests locally and in Docker within a (Linux) Ubuntu-based container. The only differences found were the fonts rendering differently in each different environment.

4.2 Puppeteer

Controlling browsers, tabs and the actions needed in a VGT framework are dependent of a browser automation library, Puppeteer. A preset framework Jest-puppeteer has premade configurations for Puppeteer to work with Jest, which makes the browser automation configuration a lot easier than creating the configuration based on Jest and Puppeteer

documentations. Puppeteer and Jest work together via web-sockets and with the help of NodeJS. (GitHub 2020)

Since Puppeteer runs on Chromium and communicates through Chrome DevTools Protocol (CDP), the protocol makes complex functionalities possible and Puppeteer works as an interface for CDP methods. In this thesis work CDP and its many use cases have been found essential for the built VGT framework to be able to work properly. (GitHub 2020)



Picture 1. Overview of Puppeteer (Puppeteer 2020)

Puppeteer documentation is comprehensive, filled with examples and problem cases (GitHub 2020). Having an encompassing documentation in a software framework, is a must have. Great documentation helps developers to understand better how each method can be

used and what's the relation to other methods. Insufficient documentation results in frustration and slowness on development. Being opensource a lot of the most common problem cases have already been solved and are easily findable via any search engine. Puppeteer offers multiple ways of doing the same thing, which makes the documentation even more important. When a framework has multiple ways of getting a similar outcome, the understanding of the framework must be at a high level. (Peng et al. 2018)

Puppeteer functions in a Chromium environment, which makes the understanding of the framework a lot easier. Each method and command run in a Chromium environment and is affected by the basic rules of Chromium. Since there's no need for additional browser support from the company's perspective, Puppeteers is a great selection for this thesis work. Puppeteer offers flexibility through CDP and through its own well documented methods. However, in very complicated scenarios scripting with Puppeteer can become very complex. Google is a huge software company, with a ton of resources. The selection of Puppeteer was based on maintainability, which is done partly by Google Employees. Having live maintainers in an opensource project is a huge factor. Live maintaining makes the project more popular and more developers will trust a project by its popularity making it even more popular and most importantly add longevity to the framework. Puppeteer's usage has grown tremendously from the year 2017. (GitHub 2020) Puppeteer is officially supported by Google's DevTools team, which makes the support more credible. The sophisticated knowledge of Chrome in the DevTools team is also a big factor, which makes developers trust Puppeteer even more.

Puppeteer has also some drawbacks. The main drawback is that Puppeteer supports only Chrome and Firefox browsers. Having limited browser support causes problems when testing websites that draw traction with different browsers than the supported ones. Visual testing assurance with Puppeteer does not cover the website quality when used with other browsers. One of the biggest issues in web development is multiple browser support. In quality assurance as web development becomes more troublesome when there are multiple browsers, so does quality assurance and testing. (Bajammal & Mesbah 2018) Styles may differ from one browser to the others and they can't be caught with Puppeteer. However, style bugs and faults, which are found with Puppeteer will probably happen in other browsers also. It takes tremendous effort to create similar visual testing tool with a framework like

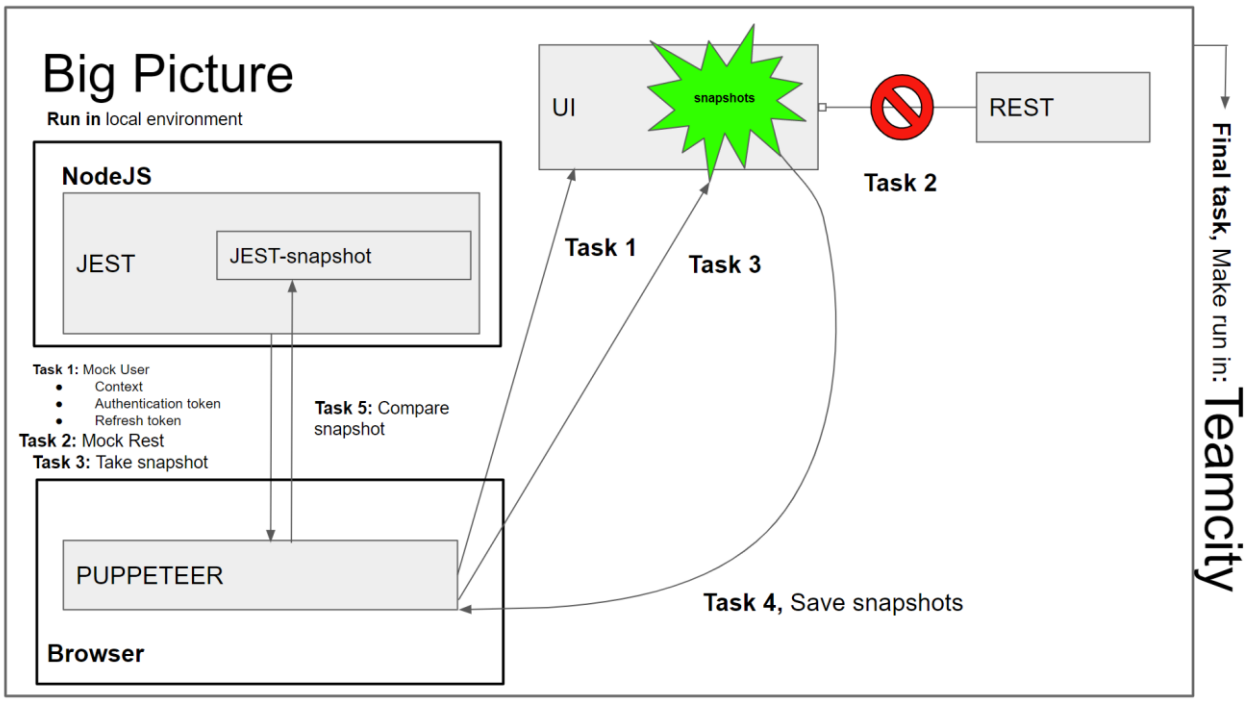
Selenium, so in the scope of this thesis the multiple browser support drawback was not seen as too critical. Also, Chrome is the most used browser by a significant share 63.62% of all web usage, hence the value of assuring Chrome-based environments is huge (StatCounter 2020).

4.3 Selected testing framework (Jest, Jest-screenshot)

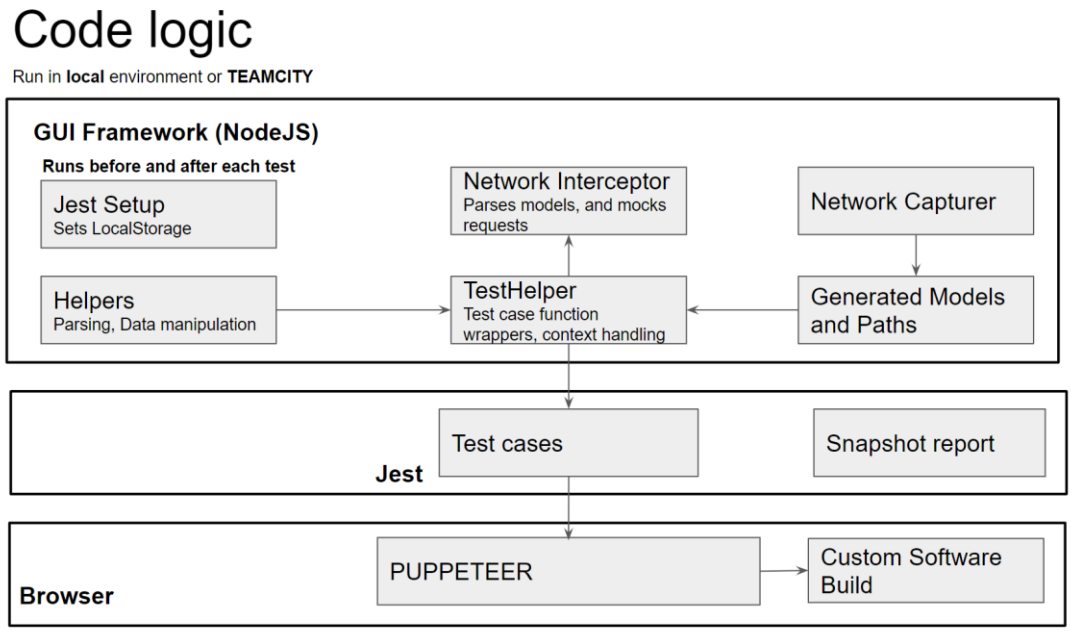
Jest is an open source JavaScript testing framework (GitHub 2020). Jest is very popular among developers and it is maintained by Facebook. Jest has resplendent documentation and excellent website to support it. Jest offers the usual configurations and methods for testing like all other modern web-development frameworks. With Jest one can define tests and run tests to get the results whether the tests have passed or not. (Jest 2020)

4.4 Implementation

The actual implementation started from assessing the requirements of this thesis project. The overall goal of this thesis was to create a VGT framework and a tool to assure the visual quality of the software product, which the company is developing. After planning and creating the tickets for this project. The work started by initializing the environments and re-assuring that every known detail was correct.



Picture 2. Overall view of the thesis project.



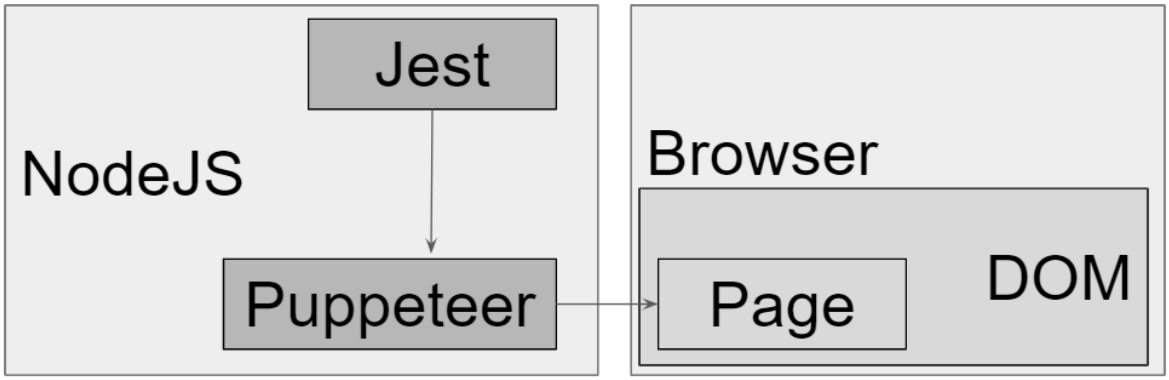
Picture 3. Overall view of the code logic.

4.4.1 Authentication

The first task of this thesis was to make the framework take a picture of a view. To make screenshotting happen a Puppeteer script was written. The script had only few lines and was rather simple. The script opened a browser and a page, navigated to a specified URL, took a

screenshot in a browser and saved the screenshot. The script worked and the screenshot looked fine. From the planning, it was known that the software system changes with time. A screenshot taken a day ago differs from a screenshot taken today. The data changing required the framework to mock the environment and data. It is also important to notice, that the written scripts need to handle inconsistencies based on browser rendering and network latency, which were one of the most troublesome aspects of the creation of this VGT framework.

When mocking a software environment, the first thing to do is to get a clear picture of how the frameworks work and in which context. Jest works in NodeJS context, but the tests are executed in browser context. To understand the execution flow of the visual testing framework, a brief explanation is needed. Jest has the highest controls in the test framework. Even in a simple test case Jest controls the flow from top to bottom. In the case of this thesis related tests, the tested software must be fooled to think, that the test framework has authenticated successfully. If no authentication is found, the system will sign the session out resulting the test framework to fail.



Picture 4. Framework context relation chart.

The software system under tests, has two implementation options to mock authentication. The first and easier one, is to control the browser to input real credentials. This method requires the knowing a user credentials and login elements. The element selecting and inputting credentials is a slow and clunky way of mocking the authentication. When thinking of authentication mocking, it is more important to understand what the system expects rather than doing the obvious. In this case the system wanted the user to have a context token,

a refresh token and an authentication token. A better way to mock the authentication was to add the required context and tokens to local storage using Puppeteer.

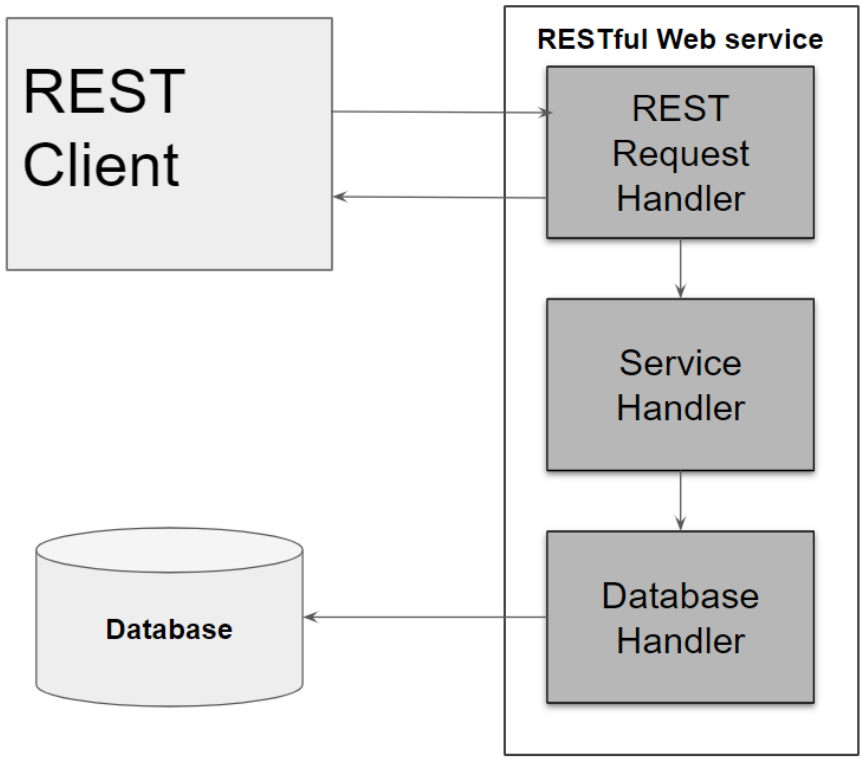
To understand any system for serious testing purposes, one way is to reverse-engineer the system (Teliskova et al. 2018). In this case having access to the source codes of the software system one could fiddle with the code and see, which code blocks were affected by which code change. This resulted gaining the necessary knowledge of the system authentication. Making decisions on a system testing framework based on source code knowledge poses risks. However, in this thesis the source code knowledge was isolated only on the authentication process mocking, hence making the source code-based decisions relatively risk-free.

Local storage is a browser-based storage, which has cookies and tokens for every page one visits. Local storage usage can be very helpful if user has a need for saving view states, for example a section might be collapsed or hidden. Local storage holds the values of the site and even during navigation. (Jemel & Serhrouchni 2014) Using local storage to set a pair of tokens and system context is a neat trick, but server still needs to validate the tokens and the system context. If the server finds the values being incorrect it will send a response to the client, which will then log out the user eventually failing the test suite. To mock the values for server and client is impossible, since it requires a real login. Fortunately, in this thesis project the need for real REST API is not needed as it may have issues and requires a testing of its own (Segura et al. 2018). All communication between the client and the real REST API is mocked. When the validation is done only on the client side of this system, then the VGT framework will work with the local storage mocking implementation successfully.

Local storage mocking is based on mirroring the real behavior of the system. The idea is to first track the real behavior and the recreate it using automation. The real implementation followed converting the real local storage keys and values to a readable JavaScript objects and then creating a method for the VGT framework to access the object and send it through Jest and Puppeteer to the Browser before anything else happens.

4.4.2 REST mocking

Mocking means faking real data with fake data. RESTful architecture is explained in the following diagram.



Picture 5. RESTful architecture.

REST is mocked by Puppeteer. Puppeteer offers a method, which intercepts network requests and gives developer a chance to abort, continue or return a custom request on behalf of the captured request. (Puppeteer 2020) The VGT framework works with intercepting the real requests and then returning custom requests to each one. There are multiple ways of generating fake requests. The challenge in this thesis work is that, there is a specific data model for each REST route. To generate fake requests, two ways were evaluated and tested. The first one was swagger-data-generator and the second was Puppeteer-based network request capturer.

Swagger is a framework which uses another framework called faker to create “dummy data” based on REST model documentation (GitHub 2020). Faking test data is a common need in the software industry since, a similar framework called Fuzzinator was presented in the research by Hodovan & Kiss 2018 verifying the claim. The company had already created a

custom solution based on Swagger and faker.js. The custom solution generated fake REST data according to the REST documentation. The problem was that the generated data was never "really" paired with the client. When trying to pair the auto generated data to the client, the client threw errors or failed. The problem was that the Swagger uses REST documentation as the baseline for fake dataset generations. In this case REST documentation was not up to date and the faked data caused the system to fail. The documentation was still usable for other purposes than Swagger. The data models were just extremely complicated. Swagger can be used to create relatively working fake data, which requires to be polished by hand. In this case the polishing was too time consuming and should be repeated if the REST routes or the data models change, and they did. The amount of changes in REST models even during the project initialization made the usage of Swagger unusable for being extremely time consuming.

Puppeteer can intercept network requests, but it can also affect how the network requests are treated. The idea is rather simple. Puppeteer opens a browser and navigates to a specific URL. When the REST responds, the REST route and data model are written in a file as a JavaScript function. The function returns a set of routes and data models, exactly like in real scenario. The fake data generator is copying real responds and later with the VGT framework usage, serving the responds to the client with the help of network interception and real copied static data. The usage and understanding to build a static "mock server" datasets, required understanding of the REST architecture.

4.4.3 Jest test sequencing

The software system under tests has different organizations and each organization has unique data and must have matching user signed in. The system checks that, which user and organization is using it on the client side. This means that the local storage must be in place before anything else happens.

Jest has a way to setup specific script execution before anything else happens. First the framework opens a browser and sets the current mock organization and user with correct authentication tokens to local storage. In the second phase of test setup, browser native navigation is called once to get correct domain in place and history API available. In the third phase the framework starts to execute tests sequentially in a specified order. After each

test the local storage is wiped clean and reset with default organization values. After the tests have been ran, teardown will start and closes the browser. The phases use methods such as “Global Setup”, “BeforeAll” and “AfterEach”. These methods do what they are named for and are used to control the flow of the tests.

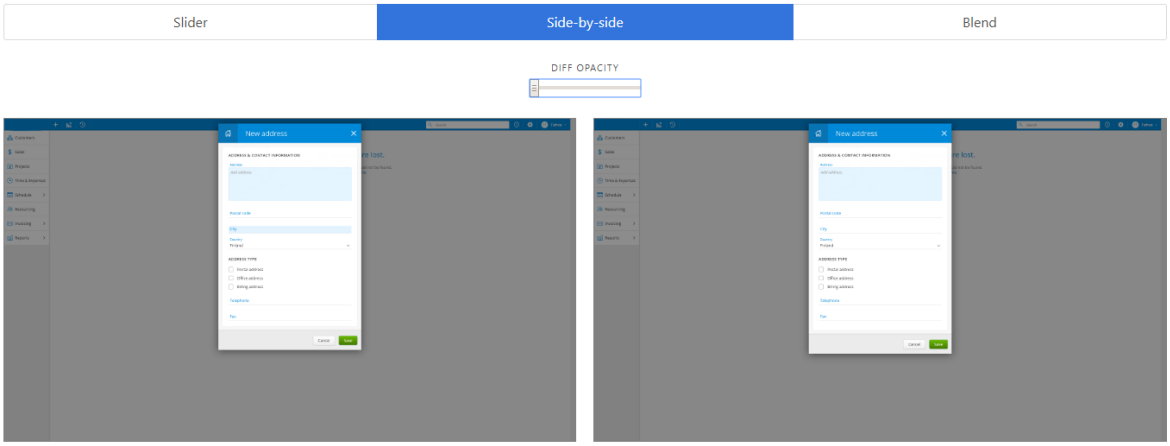
4.4.4 Jest, puppeteer & screenshot frameworks

In the VGT framework, the most important factor is to get everything stable, including the used frameworks. The reason for stability is to verify, that the visual testing framework can be extended on a firm basis. This thesis work has multiple layers that are required to understand for being able to say whether the visual testing framework is stable or not. The layers in this thesis are the company’s software that is under the tests, custom environments where the company’s software runs, multiple pre-created frameworks, custom code and configs and a REST API.

To have a solid understanding how the created VGT framework works, is to break the project in smaller pieces and focus on one piece at a time. The strategy to assure that every piece is working, is to test each piece on its own and then move to the next one. In some cases, the piece of the visual testing framework was not doing what was required and then it was changed to a better solution. When software is getting improved by rewriting functionality, it is refactored. This thesis went on multiple iterations of refactoring and assurance, until everything was stable.

4.4.5 Jest-screenshot configuration

In this thesis the visual assurance is the key element. After having the other parts of the framework in place, it was important to handle the configuration of the visual testing part. In screenshot comparison done pixel by pixel, even the smallest change can cause the test screenshot to fail.



Picture 6. HTML Report of pixel differences.

In software web-based applications there are inconsistencies based on JavaScript rendering, animations and network speed. To mitigate the risk of tests being false-positive, a configuration was created and assigned to the screenshot-method. The configuration options are displayed in Table 3.

Parameter	type	default
`detectAntialiasing`	boolean	`true`
`colorThreshold`	number	`0.1`
`pixelThresholdAbsolute`	number	`undefined`
`pixelThresholdRelative`	number	`0`
`snapshotsDir`	string	`__snapshots__`
`reportDir`	string	`jest-screenshot-report`
`noReport`	boolean	`false`

Table 3. Jest-screenshot configuration options. (GitHub 2020)

4.4.6 User-context based settings

The software system has a lot of different settings, which affect how the view is being rendered. A certain change on a setting may cause faults in a view that the setting is not used for. The way for the visual testing framework to change settings dynamically for wanted test cases, was to add the setting values to local storage. The assignment of a certain setting may

cause faults in a view that the setting is not supported for. The test cases created were addons and access rights. Addon tests should show more options or additional features compared to the view without addon setting enabled. Access right tests should restrict or show more aspects of the SUT.

The dynamic changes in local storage caused the visual testing framework to fail if tests were running in parallel. The system reads local storage values and expects them to be correct for a specific view, a dynamic change may cause a change during the read process or add additional reading. Changes during the read process or additional reading will make the authentication mocking or the network requests return unwanted data, eventually making the VGT framework unstable. When the tests are run sequentially, the VGT framework works fine.

4.4.7 Date mocking

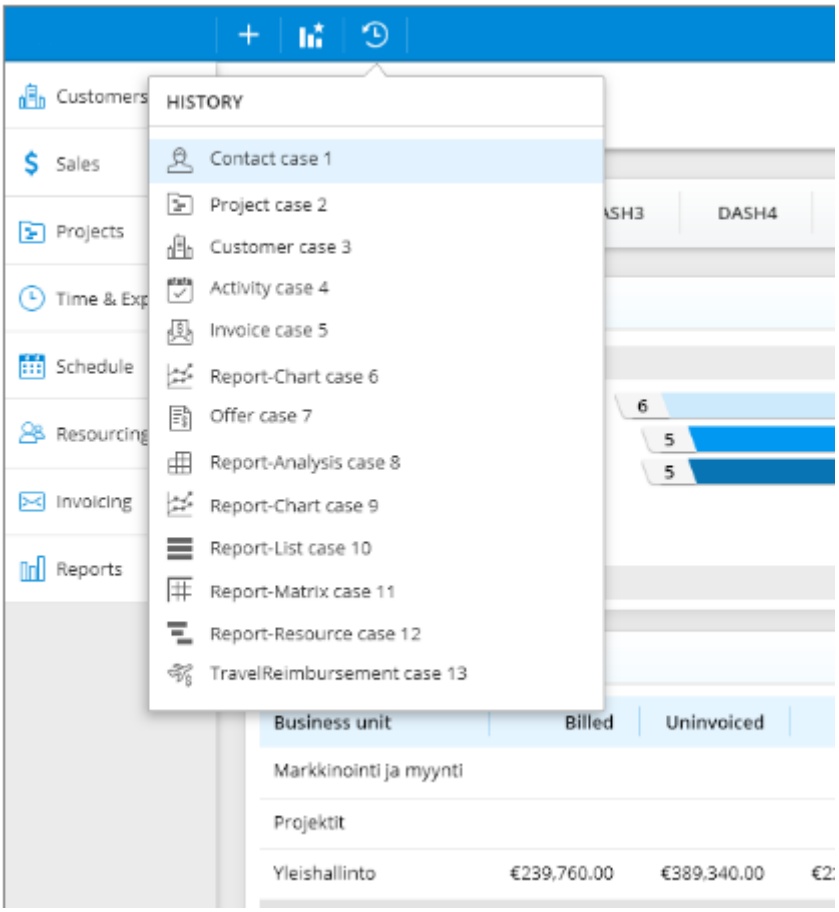
The software system is dependent on current date. In the source code of the system, a native JavaScript method called “Date.now” is called. The method returns current date and the result is used to show or calculate a date related visual element. When tests are running daily, the date related visual elements will change and cause the tests to fail even when new code has no effect on the failing view.

Additionally, the date is used in the communication between REST API and the system client. The need for all states of the system being static was imminent, no matter what the current time is. To create a mocked date in a browser context, is to run a script that overrides the native JavaScript method Date to a mocked Date object. Now when the system asks for a Date with “Date.now” -method, it will call the overridden method, which returns always the same constant specified time. The mocking is done by the native JavaScript method and running a script as soon as a page opens. This way the date value is always static and won’t cause any false positive test results.

4.4.8 Views missing URLs, click support

The visual testing framework is based on using URLs to navigate to a specific page and take a screenshot or compare it if an existing screenshot is found. The company’s software system does not have URLs for each different view, which causes the framework to have external

scripting to support views without URLs. To fix the issue, Puppeteer has methods to find elements in DOM and operate with them. The missing URLs were provided with click support. In the framework before the screenshot is taken, a clicker function is called, which clicks an element in a page. A click in the view causes the view to change from the original state resulting in a different picture.



Picture 7. A click test result.

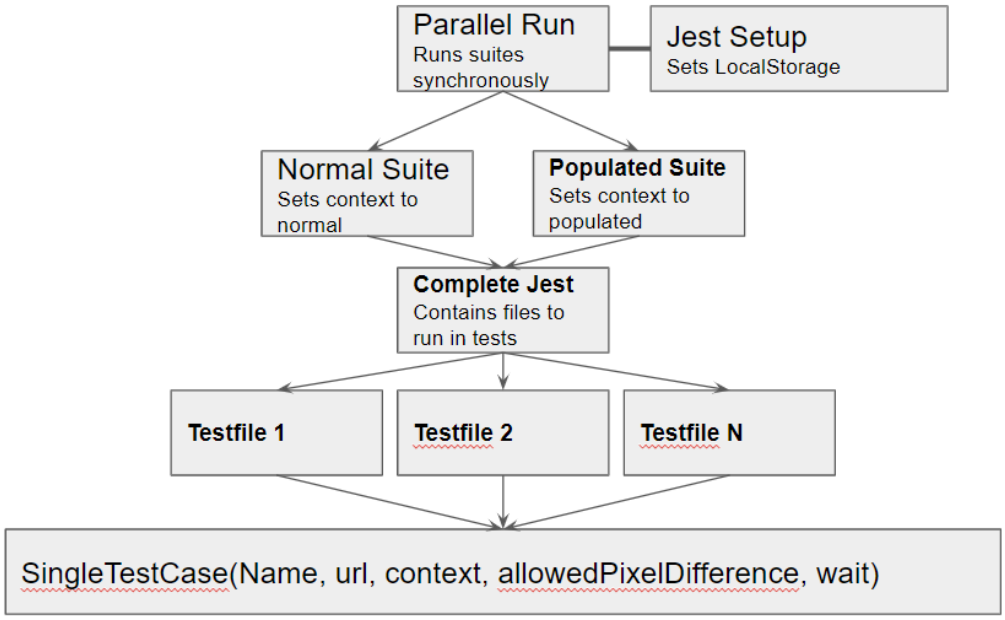
Some clicks will cause a call to REST API, these calls must be mocked in a similar way as the other calls are mocked. If a click causes a call from REST and there is no mocked data found, the visual testing framework will fail. A support was created for click caused REST models to be mocked.

4.4.9 Tests sequencing, suite architecture

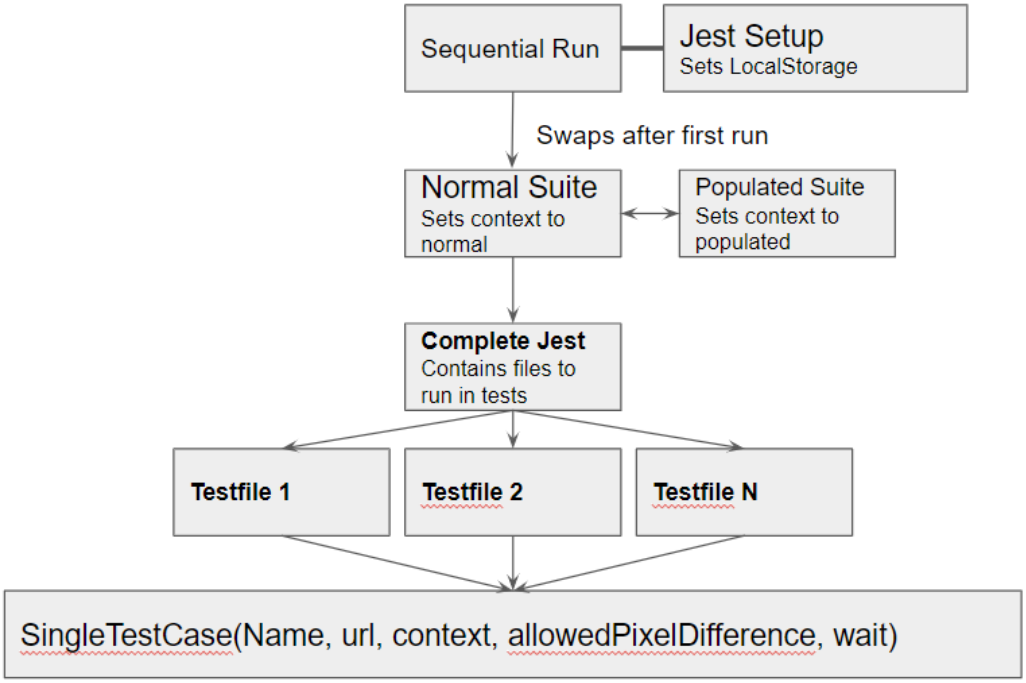
The tests are designed to be re-usable for each suite. When the test run starts and setup face is done, the first of the two data sets are selected. Then the whole suite of tests is run for both

data sets in the selected order. Both suites use the same set of tests and test cases. The outcome is still different, since the data in the screenshots is based on the data sets.

Jest supports parallel and sequential test runs. On sequential test runs, the tests are running as single test at a time. In parallel test runs, multiple tests can be run at the same time, reducing the time taken to complete the test suites. Running tests parallel with the default configuration caused issues, so the tests were separated in two suites which could be run at the same time. In the configuration of VGT framework, the performance aspect of testing resulted more complex solutions due optimizations. Through the performance optimizations a set of new issues emerged which made sequential test runs more attractive and ultimately being selected as the choice of testing runs.



Picture 8, Discontinued Parallel test stucture.



Picture 9. Sequential test structure.

4.4.10 Server-side static values

When a call is done to REST from the client, the logic running in the server side may return a value that is affected by the current date, depending on the route and the REST data model. Even if the date is mocked in the client side, server date is not. The problems arise when the REST base models are generated with Puppeteer, the exact same models that are used in the framework to mock the data. The server's date values are mocked dynamically on model generation by adding custom matchers to the model identifiers. In other words, the mocking happens by using the REST routes as keys to map the values correctly. In a case where the server-side data-based REST value changes, the static server mocking must be refactored, resulting in increased script maintenance work. Luckily, the amount of server mocks needed were limited only to 10% of test cases. Even if the server mocks would be forgotten to be maintained, the framework continues to work but making the baseline images change on a new REST model generation run.

4.4.11 Performance measurements

When tests are running in a command line environment, the time a single test takes, is not too relevant if the amount of test cases is low. However, when the amount of test cases is high, the time one test case takes is multiplied by the test case count. In the current state of

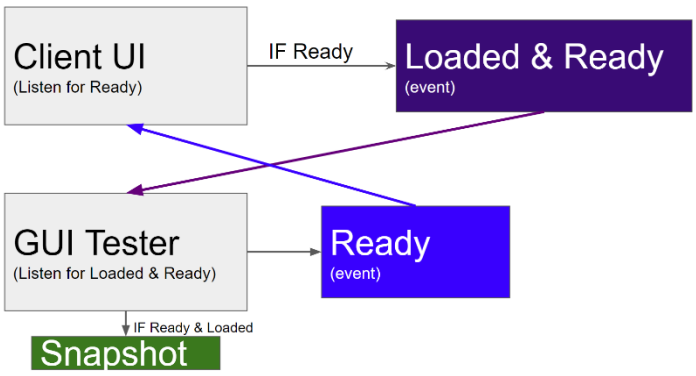
the framework usage, there are 294 test cases meaning 294 screenshots. A need for performance measurements emerged.

Performance was measured using NodeJS's native methods. The measurement results indicated that the test bottleneck was on navigation. Initially each test opened and setup a new page to test, which took around 2000 milliseconds to execute. Using DOM history API and re-using the same page instance, navigation performance went up to 10 milliseconds. With the improvements, the tests are running in a significantly efficient rate.

4.4.12 Inconsistencies

After the visual testing framework was seeming stable, consecutive runs had 1 or 2 tests failing. The fails seemed random, but after creating a tool that runs the whole test suite continuously, a pattern on test failures was found. The problems started to emerge when network was set to mimic 3G network by throttling the connection. Throttling the network caused the failing tests to fail on consecutive runs, which verified that the issue was on network loading speed.

To fic the issue of random failures, the client and the VGT framework were put to communicate to each other. A file in the source code of the system was created, which offered Puppeteer a method to expose to the client. The method marked client as ready for the framework, which made consistent screenshotting possible. In the picture 10, a more elaborate way of the communication is being displayed.



Picture 10. How the communication works between client and framework.

4.4.13 Docker

Docker is a product that enables developers to create and test applications in a safe and static environment. What Docker does, is that it can be understood as a box, where software runs. A box where everything including code, libraries, runtime, framework and devices can be configured and ran. This makes Docker great and helpful, yet it takes time and effort to understand how Docker and the containers aka. boxes work. (Marathe et al. 2019)

Docker is a great tool, but it doesn't always fit to all needs. In this thesis work, the problems of networking and Docker's problems with Windows based environments resulted in dropping Docker out of the implementation. Docker requires a lot of resources to run in Windows environments and the current support for the designed VGT framework was not at the required level, Docker was removed from the stack and replaced by TeamCity, a cloud-based CI-environment solution developed by JetBrains. The implementation now tackles font rendering and static environments by running the tests in Cloud. Cloud offers efficient testing and helps to get tests results even when multiple different environments are used on development (Arora & Dixit, 2018).

4.4.14 GUI Framework tests

A tested software has a longer lifespan, since it can be easily maintained. When tests point the problems to developers, the time consumed to the maintain the software is exceedingly lower. (Havrikov et al. 2017) The visual testing framework needed tests to ensure, that if a breaking change happens, the time taken to fix the framework should be as low as possible.

The tests were written and defined in JavaScript using Jest. Jest can be configured to have different suites in the same file level, but as it quickly turned out, the framework tests are better to isolate from the file level of visual testing tests. (Jest 2020) The framework tests included the main functionalities of the visual testing framework. The functionalities were, mocking values to browser, mocking generated values from server to files and ensuring that the parsing logic works as expected.

4.4.15 VGT Framework refactor

After almost 4 months of development the visual testing framework was merged to the company software application. Usually each feature in a software must be reviewed to assess that no bad code ends up in production (Ueda et al. 2019). Due to the nature of the development of this project and the rush that the development team had, code reviews were limited to only major features or during the meetings.

After the merge request had gained record breaking amount of comments, a need for a final refactor of the visual testing framework was needed. A day was taken to go through each method and function to understand how the framework works from a different perspective than the author's and what it really does. Almost each file and each method were renamed and the functionality got simplified.

4.4.16 Extending the implementation time

Unfortunately, the original estimate of the time this thesis as a software development project exceeded. It is quite common that software projects estimates are wrong based on various factors (Thirasakthana & Kiattisin 2018). In this case the inexperience of project creation and VGT knowledge resulted a need to extend the implementation time. There were two options offered to the company, the first was to end the project and discontinue it, meaning that the results of the thesis would be just learning related artifacts. The second option and the selected one, was to extend the implementation time and leave the project at a successfully finished state. The implementation of this thesis was granted more implementation time, which resulted ultimately finishing the GUI Tester project.

4.4.17 Integration to daily development

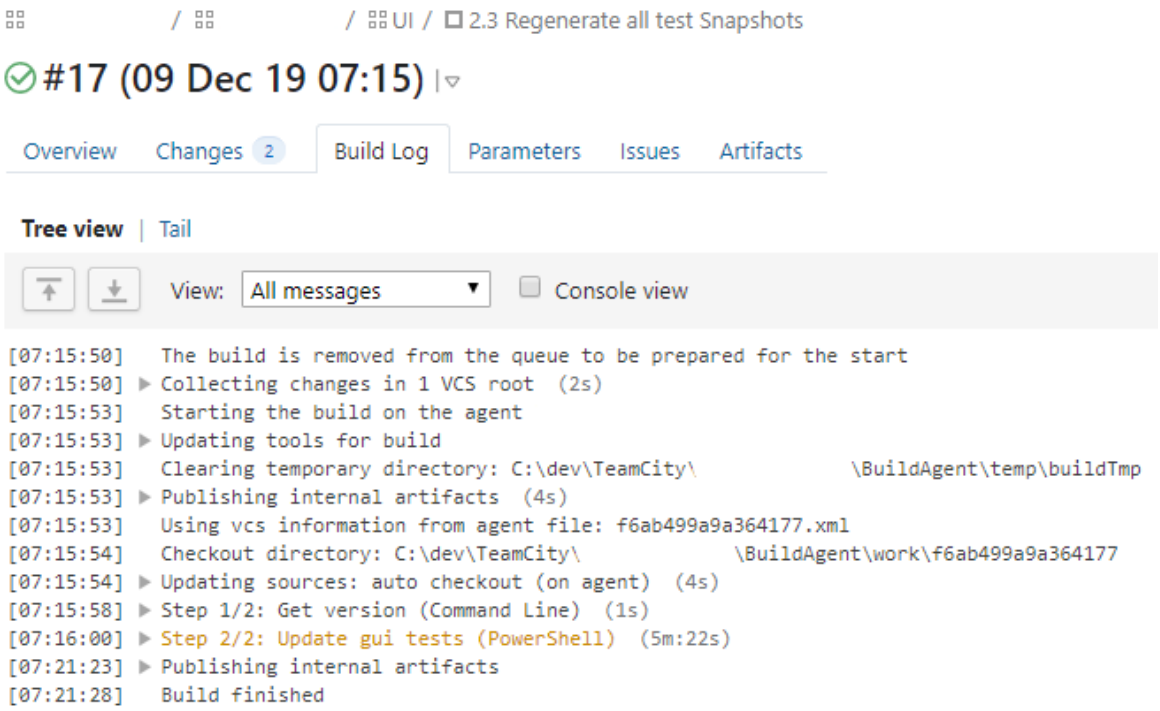
To integrate the GUI tester to daily usage required to understand how daily development works. In this company, the daily development followed industry standards. Each developer is assigned a task to complete. Each task can be a bug fix, improvement or a feature. The tasks are completed by first creating a remote branch, pulling the branch locally and developing the changes. After the changes are completed, the changes are committed. A commit hook is triggered with each push to the version control. In the hook, development

team adds any number of automated quality assurance tools to ensure the wanted quality. In this case, the commit hook already had, a style check by ESLint, unit tests running with Jasmine and Karma and a circular dependency using Madge.

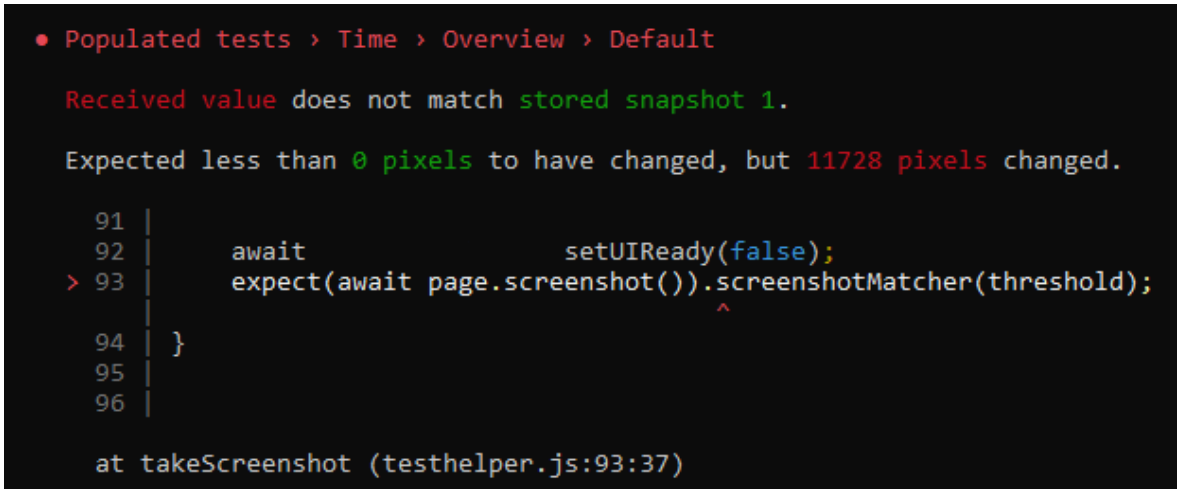
The GUI Tester, a test suite created with the VGT framework, was added to the end of the hook. The GUI Tester was tested with the commit hook and the output of the GUI Tester did not show in console, which resulted the usability of the framework blunder. After debugging and trying to locate the problem of outputting the data stream to console, the fix was discontinued. The reason for the output not working was too complicated to fix. The reason was that, the GUI Tester was running Jest tests in a Puppeteer controlled browser in a Docker container, which redirects the output to the commit-hook package and then displays the logs on a console. Somewhere between this sequence, the output stream is broken. The commit hook integration was removed. The problem between different environments producing different results on GUI Tester runs was solved with docker at first. After the failed commit hook integration and Docker's immense use of resources, the whole GUI tester project was reassigned to be run in cloud, producing always the same results as the environment is static. This way the commit hook integration and Docker were not necessary for the project to produce visual quality assurance on daily basis.

After a commit is successfully passed the commit hook, the changes are moved forward in the Pipeline. A developer can promote changes to staging environment, a test environment exactly like production environment. Before the changes enter stating environment, a TeamCity agent is triggered and the GUI tester tests start to run in the agent computer, a static environment. TeamCity is an application, which offers a way to build and automate deployment pipeline (JetBrains, 2020). The GUI Tester can be run locally and on TeamCity environments.

Picture 11. GUI Tester actions in TeamCity.



Picture 12. GUI Tester build in TeamCity.



Picture 13. Local test run image comparison failure.

Picture 14. Local test run, overall test view of settings tests.

4.4.18 Documentation

The importance of documentation is valid even on agile companies (Stettina et al. 2012). To understand the framework and how it can be used required some level of documentation. The documentation of the VGT framework had multiple iterations due to the many changes

in the framework. The documentation had a general explanation of the framework overview and how it can be used. Test examples, rest model generations and TeamCity usage are also documented. The documentation is written in Markdown format, so that the code and technical aspects are easy to read.

4.5 Implementation and maintenance

Testing suite maintenance is based on daily monitoring and test script maintenance work, when needed. Testing suite maintenance is affected by REST and authentication implementation code changes. Whenever a breaking change happens in the test suite or in the VGT framework, a report is generated by VGT framework built-in error handling and a maintainer is notified via TeamCity.

4.5.1 Continuous development

The framework was created to be extended and used. New features and problematic cases can be tested with ease of the VGT framework. Puppeteer offers a vast collection of different methods for measuring a web application. With Puppeteer one could measure the performance of the page and API calls for example in addition to the current visual assurance. Different data sets can be added to the framework with moderate amount of work. A new data set could have zero data or very overflowing data. Creating a new data set will not require any changes in the past test suites. Just by adding a new test file with new generated mock data will be enough due to the modular nature of the framework.

Web applications are built to be responsive. With the rise of mobile usage and different viewports, it is important that the software offers ways display all elements of the software regardless of the viewport. (Voutilainen et al. 2015) The drawback in the increased dynamic content of web applications is the vast amount of errors in the front-end of the applications (Durieux et al. 2018). With Puppeteer the changes of viewports can be easily tracked by adding a different config file for different viewports and passing that information to Jest running the test suites.

4.5.2 Resources

The framework requires the dependent environment to be available, whether the tests are running in local or TeamCity environments. On local runs, the tested software must be available and running in the background from CLI using Gulp.js. To use the framework, a developer only needs NodeJS installed, SUT and the GUI Tester project to be cloned from company's version control system.

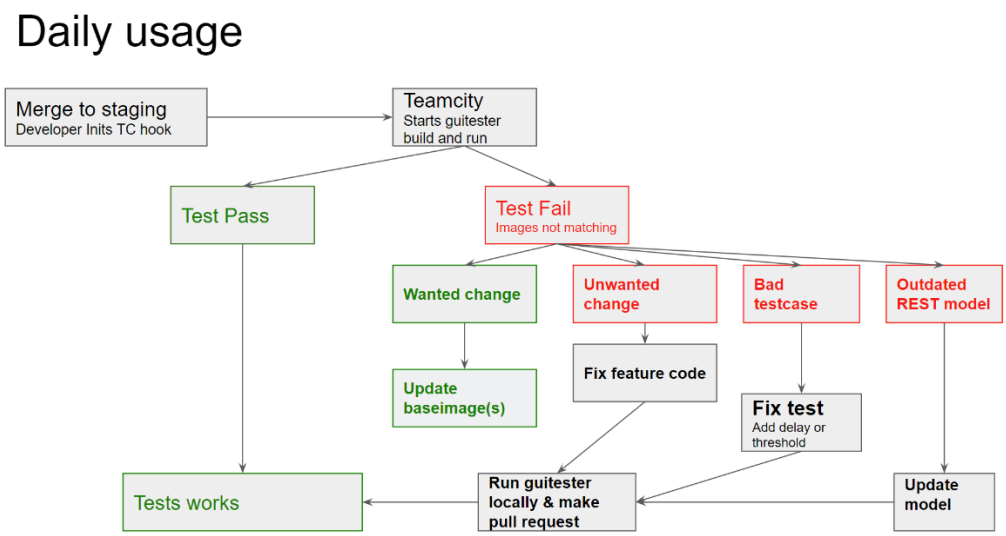
5 RESULTS

This thesis resulted in the company having implemented an automated testing tool assuring the visual qualities of a single software product. The assured product is mainly affected by the changes of a 5-member UI team and less by changes of a 5-member Backend team taking care of REST. The product is used by a vast customer base resulting in significant revenue in Tens of MEUR. The Project had assured the visual quality of the software for a month in production and not a single visual bug ended in production arguably. This was a great result, since the development team works daily producing new UI features, possibly breaking the older UI layout. The GUI tester system runs 50 times a week on average, assuring that no layout breaking UI code ends up in production. However, the GUI tester system is dependent on developers to correctly check the results of a test run before pushing the changes to production.

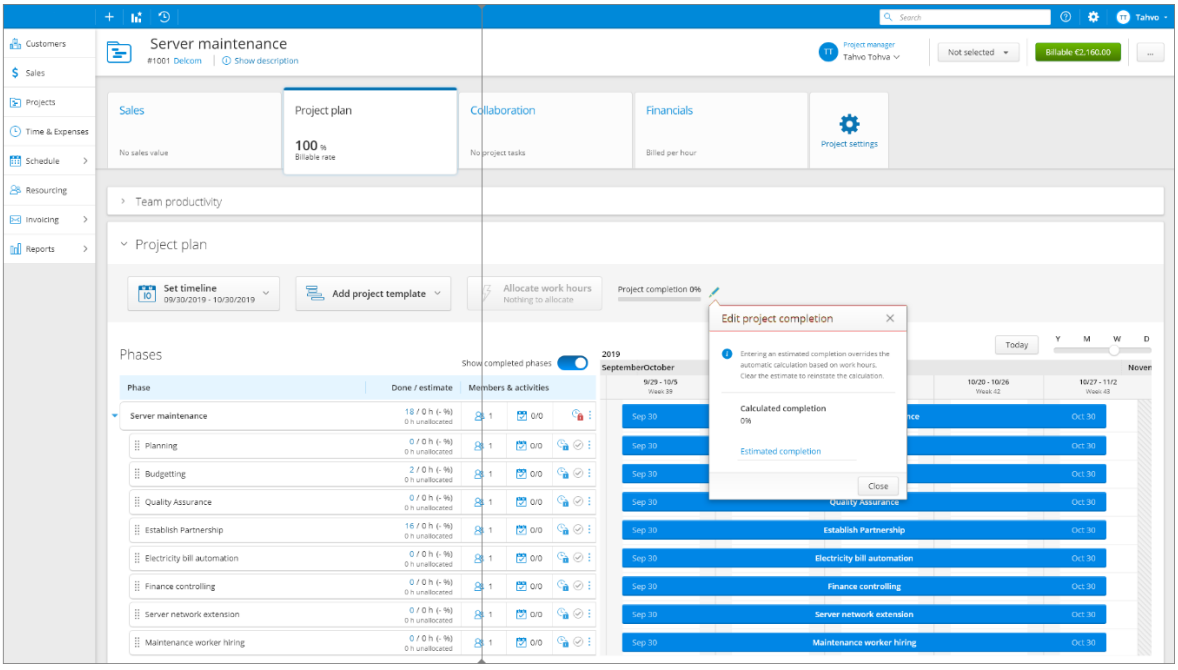
Since the testing framework was added to the daily tasks of the UI team, a training had to be organized. During the training, the GUI Tester project was used by each member of the UI team and assigned to write few test cases. The whole team was successful in getting the project work on their environments locally and accomplished writing test cases. Some minor issues in the VGT Framework usability and documentation were found and fixed. The overall documentation of the VGT framework was upgraded to ensure the usability longevity of the VGT framework. The senior developers in the Team can train new developers joining the Team for script maintenance, test generation and test usage with the help of the documentation. A training program was not seen too relevant to develop from the company's perspective, since the GUI tester project will add value even without everyone knowing how to maintain it. During the training also a rare fault was found and reported to the project management software, which made the project correctly show real-life industry quality assurance value.

The daily usage is automated by a Powershell script running in the TeamCity CI environment. The task triggers on each code change pushed to staging environment and starts to run the visual assurance test suite. If the test suite completes successfully, no actions are needed. When the testing task has not completed successfully, an automated HTML

report is generated and made accessible through internal network file sharing. A developer can open the report from a link generated in the cloud and observe the pixel differences through a visual bug reporting tool. If the fault is a wanted change, the baseline update task should be used to update the wanted change as a new base-image. Otherwise a developer may want to check the code changes, fix the test case or update the mocked REST models. If a developer chooses to make changes, the GUI Tester project can be used locally to test and verify the changes before pushing the changes to staging environment.



Picture 15. Daily Usage of GUI Tester



Picture 16. Visual HTML report tool displaying UI changes.

From an academic point of view, no prior research of Jest and Puppeteer based VGT framework has been published. Research about Puppeteer and Jest based projects are also covered little to none in published academic papers. This phenomenon can originate from Puppeteer and Jest being a recent addition to the open source testing and browser automation toolbox. Puppeteer's first documented version is marked available from late 2017, which makes Puppeteer age just over 2 years. Jest is also a quite new addition, published in the summer of 2016 to GitHub as JavaScript testing framework. (GitHub, 2020) Due to the freshness of the technologies used in this thesis work, the work acts as an example of state of the art -research in web automation testing and in automation of visual quality assurance tasks. This work acts as a proof of successful VGT system, which is integrated to daily usage in industry practice with the latest technologies publicly available. The current indication of added value in visual quality assurance from the developed VGT framework is satisfactory. However, the approximate longevity of the project and results in a yearly basis were not available as this thesis work got finished.

With multiple development iterations and discussions, the GUI Tester project was finished. A VGT framework is integrated to daily usage to offer a way to write and run tests. The project consists of a test runner, a mock data generator and a testing framework. Together the selected technologies make up the GUI Tester project. The project was created by getting the selected technologies, packages and frameworks working together and produce a way to write tests and run tests. The company's visual testing needs of one of their software products were filled sufficiently with the creation and usage of the technical part of this thesis work.

6 CONCLUSION

In this chapter the conclusion, reliability and possibilities to extend the work are presented. The key deliverable in this thesis work is the created GUI Tester project and the VGT framework. The key findings of this thesis are shown in the implementation process of the VGT framework, even if a project, a piece of software or a dedicated visual testing framework has a simple purpose, the relations to other technical interfaces or to humans can end up making the created deliverable unusable. In this thesis the risk of creating a framework that would be no use in the industrial practice was high due to the implementation time frame. In the following sections the details of the key findings and deliverables are discussed.

The Results of this thesis indicates that the creation of visual quality assurance framework does have challenges but is possible and worth of doing, by both academic and industrial settings-wise. A framework should be created if a software requires a custom solution for screenshotting and assuring the visual quality of a software including a need to fake a part of the software. Considering different commercial products is also recommended, since smaller software products may not need a custom solution, depending on the nature of the SUT. Creating a project with a single framework usually has tutorials or examples and is relatively easy to copy. Creating a project with multiple frameworks working together changes the level of understanding one must have. The benefits of having multiple frameworks available speeds up the development process. However, the more external dependencies, a project has, the harder it is to locate whether a problem is caused by external factor or by the developer himself. Also, the communication problems with frameworks working together and how information should be passed required a high level of understanding about JavaScript and NodeJS. High level architectural decisions should be discussed extensively with senior developers if a project has an inexperienced lead developer. In this thesis work it was noted, that if some of the early decisions of the framework development would have been different, the outcome of the current solution would have been faster to achieve, which is true to almost any software project.

6.1 Reliability

The results and conclusions are relatively reliable. The company is using the framework and the tests are running on daily basis, resulting the visual quality of the software being assured through automation in an industrial setting. There is a possibility of human errors, misconception, measurement, research being bias and failure of interpretation.

The created GUI Tester project may have issues based on the test results reliability. False positiveness on the test results can be happen due to animations, browser rendering speed or network load. In addition, the cloud and client software must be running and be accessible for the GUI Tester to test against. Test results reliability can suffer also from unknown factors due to software aging or misidentification of found issues (Morgado & Paiva 2018).

```
[07:21:22]      [Step 2/2] Snapshot Summary
[07:21:22]      [Step 2/2]  â€º 29 snapshots updated from 2 test suites.
[07:21:22]      [Step 2/2]
[07:21:22]      [Step 2/2] Test Suites: 2 passed, 2 total
[07:21:22]      [Step 2/2] Tests:       294 passed, 294 total
[07:21:22]      [Step 2/2] Snapshots:   29 updated, 29 total
[07:21:22]      [Step 2/2] Time:        310.511s
[07:21:22]      [Step 2/2] Ran all test suites.
[07:21:22]      [Step 2/2]
[07:21:22]      [Step 2/2]
[07:21:22]      [Step 2/2] Process exited with code 0
[07:21:23] ▶ Publishing internal artifacts
[07:21:28]   Build finished
```

Picture 17. TeamCity Test run summary affected by reliability factors.

6.2 Extending research

There is a huge amount of possibilities to extend the VGT framework and visual testing research. In this thesis, the custom solution created acted as a starting path to explore the modern options in VGT research. The GUI Tester project can be extended to handle dynamically changing REST identifiers and automatic model generations on failure, making the framework more robust and stable. Also, the test suite can be extended to cover more views with different configurations including different viewports and full-sized screenshots. Assuring different viewports is beneficial for quality assurance, since the SUT application is responsive (Althomali et al. 2019).

Extending the GUI Tester to cover more applications is also a way to create more value in the field of visual quality assurance. The current implementation supports any web application with a RESTful backend architecture with minor changes. These minor changes include removing or modifying the client-framework communication, server mocking and REST-model generation. The named dependencies should be measured and refactored to be more generic, in order to extend the GUI Tester project to other web applications at ease.

Using the latest technologies including machine learning, neural networks and artificial intelligence will bring improvements in VGT. Studies have shown that using deep tech in GUI research can be beneficial, yet a relatively far away from industrial practice. (Lu et al., 2018) The GUI Tester project can be also extended to use better deep tech -based solutions due its modular nature, as being built from part readymade components.

Usability and maintainability are the key factors guiding the design and implementation decisions throughout the project. Measuring time spent on the maintenance work would be greatly beneficial to gain more valuable information about the usage of the created VGT framework. The VGT Framework usability should be also measured in a longer time span by interviewing or surveying the users to have a reliable set of issues and changes to implement.

The current plan for the longevity of the project is based on the short-term results the GUI tester project shows. If the relation between script maintenance work and assured visual quality is seen investable and valuable from the company's viewpoint, then there should be no hinderance on long-term usage of the created GUI tester. The VGT framework is being used by the company's developer team to extend the test cases to more views and states of the system, resulting improved visual assurance quality. The lead maintainer for GUI Tester was anointed, but with the current assurance statistics the value of further development of the thesis project is unknown. Currently there is no information available of a person assigned to continue extending this thesis work. However, this thesis project can be modified easily by the company's UI developers to a better visual assurance tool if needed, since the source codes has been assured and refactored with multiple pull requests with the company's UI Team members. A few discussions were taken during the technical implementation of this thesis work. The discussion topics were making this project open source and a general

solution to visual testing automation framework or extending this thesis work to assure the company's other products visual quality internally. Overall the process of extending this thesis work is still open in both ways, only time will tell whether the created project lives up to assure the visual quality of the company's product in the long-term as the project currently is.

REFERENCES

Adachi, Y., Tanno, H. & Yoshimura, Y., 2018, Reducing redundant checking for visual regression testing, 25th Asia-Pacific Software Engineering Conference (APSEC)

Adamoli, A., Zaparanuks, D., Jovic, M. & Haswirth, M., 2011, Automated GUI performance testing, Software Quality Journal

Alameer, A., Mahajan, S. & Halfond, W.G.J., 2016, Detecting and Localizing Internationalization Presentation Failures in Web Applications, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Alégroth, E. & Feldt, R., 2014, Industrial Application of Visual GUI Testing: Lessons Learned, Continuous Software Engineering

Alégroth, E., Gao, Z., Oliveira, R.A.P & Memon, A., 2015, Conceptualization and Evaluation of Component-based Testing Unified with Visual GUI Testing: an Empirical Study, IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)

Alégroth, E., Feldt, R., Gustafsson, J. & Ivarsson, H., 2017, Replicating Rare Software Failures with Exploratory Visual GUI Testing, IEEE Software Magazine

Alégroth, E. & Feldt, R., 2017, On the long-term use of visual gui testing in industrial practice: a case study, Empirical Software Engineering

Alégroth, E., Feldt, R. & Ryrholm, L., 2014, Visual GUI testing in practice: challenges, problems and limitations, Computer Weekly News

Alégroth, E., Karlsson, A. & Radway, A., 2018, Continuous Integration and Visual GUI Testing: Benefits and Drawbacks in Industrial Practice, IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)

Alíc, D., Djedovic, A., Omanovic, S. & Tanovic, T., 2017, Impact of human resources changes on performance and productivity of Scrum teams, 40th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)

Althomali, I., Kapfhammer, G.M. & McMinn, P., 2019, Automatic Visual Verification of Layout Failures in Responsively Designed Web Pages, 12th IEEE Conference on Software Testing, Validation and Verification (ICST)

Al-Tarawneh, F.H. & Althunibat, A., 2019, Pilot Study of Healthcare COTS Software Evaluation and Selection, IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)

Anand, A., Reshadi, M., Du, B., Kolam, H., Jaiswal, S. & Akella, A., 2015, A Case for Application-Managed Cache For Browser, IEEE International Conference on Multimedia and Expo (ICME)

Amalfitano, D., Nicola, A., Fasolino, A.R. & Tramontana, P., 2015, A Conceptual Framework for the Comparison of Fully Automated GUI Testing Techniques, 30th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW)

Arora, P. & Dixit, A., 2018, Cloud Testing – Proposed Framework with its Scope, Importance, Methodologies, Challenges, 4th International Conference on Computing Communication and Automation (ICCCA)

Banerjee, S., Helmick, J., Syed, Z. & Cukic, B., 2015, Eclipse vs. Mozilla: A Comparison of Two Large-Scale Open Source Problem Report Repositories, IEEE 16th International Symposium on High Assurance Systems Engineering

Bajammal, M. & Mesbah, A., 2018, Web Canvas Testing through Visual Inference, IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)

BEM, 2020, Block Element Modifier, [Accessed, 24.01.2020, http://getbem.com/]

Beroual, O., Guerin, F. & Halle, S., 2017, Searching for Behavioural Bugs with Stateful Test Oracles in Web Crawlers, IEEE/ACM 10th International Workshop on Search-Based Software Testing (SBST)

Börjesson, E. & Feldt, R., 2012, Automated System Testing using Visual GUI Testing Tools: A Comparative Study in Industry, IEEE Fifth International Conference on Software Testing, Verification and Validation

Börjesson, E., 2012, Industrial Applicability of Visual GUI testing for System and Acceptance Test Automation, IEEE Fifth International Conference on Software Testing, Verification and Validation

Coppola, R., Morisio, M., Torchiano, M. & Ardito, L., Scripted GUI testing Android open-source apps: evolution of test code and fragility causes, IEEE Transactions on Reliability

Choudhary, S.R., Versee, H., Orso, A., 2010, A Cross-browser Web Application Testing Tool, IEEE International Conference on Software Maintenance

Chowdhury, A.R., 2019, Best 8 JavaScript Testing Frameworks in 2019 [Accessed 08.01.2020, https://www.lambdatest.com/blog/top-javascript-automation-testing-framework/]

Dadkhah, M., 2016, Semantic-Based Test Case Generation, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Debroy, V., Brimble, L., Yosy, M. & Erry, A., 2018, Automating Web Application Testing from the Ground Up: Experiences and Lessons Learned in an Industrial Setting, IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)

Dobles, I., Martínez, A. & Quesada-López, C., 2019, Comparing the effort and effectiveness of automated and manual tests, 14th Iberian Conference on Information Systems and Technologies (CISTI)

Durieux, T., Hamadi, Y. & Monperrus, M., 2018, Fully Automated HTML and Javascript Rewriting for Constructing a Self-healing Web Proxy, IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)

Ebert, C., Gallardo, G., Hernantes, J., & Serrano, N., 2016, DevOps, IEEE Software Magazine

ESLint, https://eslint.org/ [Accessed 14.01.2020]

Garg, D., Abhisnek, S. & Bansal, A., 2015, A Framework For Testing Web Applications Using Action Word Based Testing, 1st International Conference on Next Generation Computing Technologies (NGCT)

GitHub, https://github.com/ [Accessed 08.01.2020]

Hammoudi, M., Rothermel, G. & Tonella, P., 2016, Why do Record/Replay Tests of Web Application Break?, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Hanna, S., Jaber, H., 2019, An Approach for Web Applications Test Data Generation Based on Analyzing Client Side User Input Fields, 2nd International Conference on new Trends in Computing Sciences (ICTCS)

Happonen, A., 2011, Muuttuvan kysyntään sopeutuva varastonohjausmalli, LUTPub

Havrikov, N., Gambi, A., Zeller, A., Arcuri, A. & Galeotti, J.P., 2017, Generating Unit Tests with Structured System Interactions, IEEE/ACM 12th International Workshop on Automation of Software Testing (AST)

Hodovan, R. & Kiss, A., 2018, Fuzzinator: An Open-Source Modular Random Testing Framework, IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)

Imtiaz, J., Sherin, S., Khan, M.U. & Iqbal, M.Z., 2019, A systematic literature review of test breakage prevention and repair techniques, Information and Software Technology

Issa, A., Sillito, J. & Garousi, V., 2012, Visual Testing of Graphical User Interfaces: an Exploratory Study Towards Systematic Definitions and Approaches, 14th IEEE International Symposium on Web Systems Evolution (WSE)

Jemel, M. & Serhrouchni, A., 2014, Content protection and secure synchronization of HTML5 local storage data, IEEE 11th Consumer Communications and Networking Conference (CCNC)

Jest, 2020, https://jestjs.io/ [Accessed 20.01.2020]

Joy, J. & Singh, D.P., 2015, A Generic Framework design to enhance capabilities of an Enterprise Test Automation Framework, International Conference on Applied and Theoretical Computing and Communication Technology (iCATccT)

Khan, T.A., Runge, O. & Heckel, R., 2012, Testing against Visual Contracts: Model-Based Coverage, International Conference on Graph Transformation

Kinnunen, S-K, Happonen, A., Marttonen-Arola, S. & Kärri, T., 2019, Traditional and extended fleets in literature and practice: Definition and untapped potential, International Journal of Strategic Engineering Asset Management

Kiranagi, V.H. & Shyam, G.K., 2017, Feature Driven Hybrid Test Automation Framework (FDHTAF) for web based or cloud based application testing, International Conference On Smart Technologies For Smart Nation (SmartTechCon)

Kortelainen, H. & Happonen, A., 2017, From data to decisions – the re-distribution of roles in manufacturing ecosystems, [Accessed 24.01.2020, https://vttblog.com/2017/12/20/from-data-to-decisions-the-re-distribution-of-roles-in-manufacturing-ecosystems/]

Kortelainen, H. & Happonen, A., 2017, Digitalisaatio muokkaa tiedon hallintaan pohjautuvien palveluiden markkinointia, [Accessed 24.01.2020, https://vttblog.com/2017/12/12/digitalisaatio-muokkaa-tiedon-hallintaan-pohjautuvien-palveluiden-markkinoita/]

Kortelainen, H., Happonen, A. & Kinnunen, S-K, 2016, Fleet Service Generation – Challenges in Corporate Asset Management, Lecture Notes in Mechanical Engineering

Lenka, R.K., Satapathy, U. & Dey, M., 2018, Comparative Analysis on Automated Testing of Web-based Application, International Conference on Advances in Computing, Communication Control and Networking (ICACCCN)

Lu, H., Wang, L., Ye, M., Yan, K. & Jin, Q., 2018, DNN-based Image Classification for Software GUI Testing, IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI)

Mahajan, S., Bailan, L., Behnamghader, P. & Halfond, W.G.J., 2016, Using Visual Symptoms for Debugging Presentation Failures in Web Applications, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Mahajan, S., Gadde, K.B., Pasala, A. & Halfond, W.G.J., 2016, Detecting and Localizing Visual Inconsistencies in Web Applications, 23rd Asia-Pacific Software Engineering Conference (APSEC)

Marathe, N., Gandhi, A. & Shah, J., 2019, Docker Swarm and Kubernetes in Cloud Computing Environment, 3rd International Conference on Trends in Electronics and Informatics (ICOEI)

Mazinanian, D. & Tsantalis, N., 2017, CSSDev: Refactoring duplication in Cascading Style Sheets, IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)

Morgado, I.C. & Paiva, A.C.R., 2017, Mobile GUI Testing, Software Quality Journal

Moura, J.L. & Charao A.S., 2017, Test Case Generation from BPMN Models for Automated Testing of Web-Based BPM Applications, 17th International Conference on Computational Science and Its Applications (ICCSA)

Npm, 2020, https://www.npmjs.com/ [Accessed 23.01.2020]

Muhamad, F., Sarno, R., Ahmadiyah, A.S. & Rochimah, S., 2016, Visual GUI Testing in Continuous Integration Environment, International Conference on Information & Communication Technology and Systems (ICTS)

Myrcha, J. & Rokita, P., 2014, Automated Visual Perception-Based Web Browser Rendering Results Comparison with Multi-part Fragment Image Matching, International Conference on Computer Vision and Graphics

Nagowah, L. & Kora-Ramiah, K., Automated Complete Test Case Coverage for Web Based Applications, International Conference on Infocom Technologies and Unmanned Systems (Trends and Future Directions) (ICTUS)

Navarro, P.L.M., Ruiz, D.S. & Pérez, G.M., 2019, A Proposal for Automatic Testing of GUIs Based on Annotated Use Case, Advances in Software Engineering

Negara, N. & Stroulia, E., 2012, Automated Acceptance Testing of JavaScript Web Applications, 19th Working Conference on Reverse Engineering

Nguyen, B.N., Robbins, B., Banerjee, I. & Memon, A., 2014, GUITAR: an innovative tool for automated testing of GUI-driven software, Automated Software Engineering

Nishi, Y., 2015, Design principles in Test Suite Architecture, IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)

Ozcan, M., 2019, An Industrial Study on Application of Combinatorial Testing in Modern Web Development, IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)

Panda, M., Mohapatra, D.P., 2014, Automated Graphical User Interface Regression Testing, Proceedings of International Conference on Internet Computing and Information Communications

Peng, X., Zhao, Y., Liu, M. et al., 2018, Automatic Generation of API Documentations for Open-Source Projects, IEEE Third International Workshop on Dynamic Software Documentation (DySDoc3)

Pham, R., Holzmann, H. & Schneider, 2013, Tailoring video recording to support efficient GUI testing and debugging, Software Quality Journal

Puppeteer, 2020, https://pptr.dev/ [Accessed 20.01.2020]

Quental, N.C., Siebra, C.A. & Quintino, J.P., 2019, Automating GUI Response Time Measurements in Mobile and Web Applications, IEEE/ACM 14th International Workshop on Automation of Software Test (AST)

Rahman, M., Chen., Z. & Gao, J., 2015, A Service Framework for Parallel Test Execution on Developer's Local Development Workstation, IEEE Symposium on Service-Oriented System Engineering

Ramler, R., Buchgeher & R., Klammer, C., 2018, Adapting automated test generation to GUI testing of industry applications, Information and Software Technology

Ramya, P., Sindhura, V. & Sagar, P.V., 2017, Testing using Selenium Web Driver, Second International Conference on Electrical, Computer and Communication Technologies (ICECCT)

Sahoo, S. & Abhishek, R., 2017, A framework for Optimization of Regression Testing of Web Services Using Slicing, International Conference on Advances in Computing, Communications and Informatics (ICACCI)

Savchenko, D., Ashikhmin, N. & Radchenko, G., 2016, Testing-as-a-Service Approach for Cloud Applications, IEEE/ACM 9th International Conference on Utility and Cloud Computing (UCC)

Segura, S., Parejo, J.A., Troya, J. & Ruiz-Cortez A., 2018, Metamorphic Testing of RESTful Web APIs, IEEE/ACM 40th International Conference on Software Engineering (ICSE)

Selay, E., Zhou, Z.Q., & Zou, J., 2014, Adaptive Random Testing for Image Comparison in Regression Web Testing, International Conference on Digital Image Computing: Techniques and Applications (DICTA)

Shariff, S.M., Li, H. & Bezemer, C-P., 2019, Improving the Testing Efficiency of Selenium-based Load Tests, IEEE/ACM 14th International Workshop on Automation of Software Test (AST)

Sharma, Y., Shatakshi, P., Dagur, A. & Chaturvedi, R., 2018, Automated Bug Reporting System In Web Applications, 2nd International Conference on Trends in Electronics and Informatics (ICOEI)

Shin, D. & Bae, D-H., 2016, A Theoretical Framework for Understanding Mutation-Based Testing Methods, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Shun, N., Komatsu, K., Tanaka, T. & Matsumoto, K., 2017, Development and Evaluation of Large Screen Digital Kanban with Smartphone Operation, 6th IIAI International Congress on Advanced Applied Informatics (IIAI-AAI)

Singi, K., Kaulgud, V. & Era, D., 2015, Visual Requirements Specification and Automated Test Generation for Digital Applications, IEEE/ACM 2nd International Workshop on Requirements Engineering and Testing

Simos, D.E., Zivanoc, J. & Leithner, M., 2019, Automated Combinatorial Testing for Detecting SQL Vulnerabilities in Web Applications, IEEE/ACM 14th International Workshop on Automation of Software Test (AST)

Sneha, M. & Gowda, M., 2017, Research on Software Testing Techniques and Software Automation Testing Tools, 2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS)

Srinivasan, S.M. & Sagnwan, R.S., Web App Security, A Comparison and Categorization of Testing Frameworks, IEEE Software Magazine

StatCounter, 2020, https://gs.statcounter.com/browser-market-share [Accessed 27.01.2020]

Stettina, C., Heijsek, W. & Fegri, T.E., 2012, Documentation in Agile Teams: The Role of Documentation Formalism in Achieving a Sustainable Practice, Agile Conference

Stocco, A., Leotta, M., Ricca, F. & Tonella, P., 2014, PESTO: A Tool for Migrating DOM-based to Visual Web Tests, IEEE 14th International Working Conference on Source Code Analysis and Manipulation

Sun, C., Zhang, Z., Jiang, B. & Chan, W.K., 2016, Facilitating Monkey Test by Detecting Operable Regions in Rendered GUI of Mobile Game Apps, IEEE International Conference on Software Quality, Reliability and Security (QRS)

Sutapa, F., Kusumawardani, S.S., Permanasari, A.E., 2019, Automated Test Suite for Regression Testing Based on Serenity Framework: A Case Study, International Conference of Artificial Intelligence and Information Technology (ICAIIT)

Takei, N., Saito, T., Takasu, K. & Yamada, T., 2015, Web Browser Fingerprinting using only Cascading Style Sheets, 10th International Conference on Broadband and Wireless Computing, Communication and Applications (BWCCA)

Tanaka, H., 2019, X-BROT: Prototyping of Compatibility Testing Tool for Web Application based on Document Analysis Technology, International Conference on Document Analysis and Recognition Workshops (ICDARW)

Teliskova, M., Török, J. et al., 2018, Implementation of Innovative Digitalization Methods in Reverse Engineering, 5th International Conference on Industrial Engineering and Applications (ICIEA)

Thirasakthana, M. & Kiattisin, S., 2018, Identifying Standard Testing Time for Estimation Improvement in IT Project Management, 3rd Technology Innovation Management and Engineering Science International Conference (TIMES-iCON)

Ueda, Y., Ishio, T., Ihara, A. & Matsumoto, K., 2019, Mining Source Code Improvement Patterns from Similar Code Review Works, IEEE 13th International Workshop on Software Clones (IWSC)

Viswanadha, K., Shivaji, S., Shanmugam, R., Song, S., Choi, S. & Kumar, N., 2019, ATARI: Autonomous Testing and Real-time Intelligence – A framework for autonomously testing modern applications, IEEE International Conference On Artificial Intelligence Testing (AITest)

Voutilainen, J., Salonen, J. & Mikkonen, T., 2015, On the Design of a Responsive User Interface for a Multi-Device Web Service, 2nd ACM International Conference on Mobile Software Engineering and Systems

Wetzlmaier, T., Ramler, R. & Putschögl, W., 2016, A Framework for Monkey GUI Testing, IEEE International Conference on Software Testing, Verification and Validation (ICST)

Winkler, D., Meixner, K. & Biffl, S., 2018, Towards Flexible and Automated Testing in Production Systems Engineering Projects, IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)

Xie, T., Tillman, N. & Laksman, P., 2016, Advances in Unit Testing: Theory and Practice, IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)

Yildiz, A., Aktemur, B. & Sözer, H., 2012, Rumadai: A Plug-In to Record and Replay Client-Side Events of Web Sites with Dynamic Content, Second International Workshop on Developing Tools as Plug-Ins (TOPI)

Yao, Y. & Wang, X., 2012, A Distributed, Cross-Platform Automation Testing Framework for GUI-Driven Applications, Proceedings of 2012 2nd International Conference on Computer Science and Network Technology

Yousaf, N., Azam, F., Butt, W.H., Anwar, M.W. & Rashid, M., 2019, Automated Model-Based Generation for Web User Interfaces (WUI) From Interaction Flow Modeling Language (IFML) Models, IEEE Access